

CMake ile Modern C/C++

Derleme Sistemleri, Bağımlılık Yönetimi ve Paketleme Rehberi

Tek Dosyadan Çok Modüllü Projeye, Sıfırdan İleri Seviyeye Kapsamlı Başvuru

Bu belge önce bir derleme sisteminin **neden gerektiğini** (derleyici, bağlayıcı, make'in sınırları), sonra CMake'in **nasıl çalıştığını** (configure/generate/build aşamaları, hedefler, özellik yayılımı) ve ardından **gerçek projelerde** kütüphane yazmayı, bağımlılık çekmeyi, test ve paketlemeyi **çok sayıda çalışan örneklerle** anlatır. Örnekler hem C hem C++ için verilir ve modern (hedef-tabanlı) CMake stiline uygundur.

Kapsam — Kısım 1 (Temeller): derleme nedir • önışlemci/derleyici/bağlayıcı • make'in sorunları • CMake felsefesi • kurulum • ilk C ve C++ projesi • out-of-source build • derleme akışı.

Kapsam — Kısım 2 (Dil ve Hedefler): değişkenler & tipler • cache • liste/string işlemleri • add_executable/add_library • target_* komutları • PUBLIC/PRIVATE/INTERFACE • static/shared/interface/object kütüphaneler.

Kapsam — Kısım 3 (Orta Seviye): alt dizinler • proje düzeni • find_package • koşullar & döngüler • fonksiyon/makro • generator expression • derleme tipleri • sanitizeler.

Kapsam — Kısım 4 (İleri): FetchContent • ExternalProject • install & CPack • kendi config paketini yazmak • CTest ile test • CMakePresets.json • toolchain & çapraz derleme.

Kapsam — Kısım 5 (Pratik): tam çok-modüllü C++ projesi • tam C projesi • en iyi uygulamalar • sık hatalar & çözümleri • komut cheat sheet.

Hazırlanma Tarihi: 18 Temmuz 2026

CMake 3.20+ • C11/C17 • C++17/C++20 • GCC/Clang/MSVC • Ninja & Make

Contents

1	Bir Derleme Sistemi Neden Gerekir?	2
1.1	Derleme Zinciri: Önışlemci, Derleyici, Bağlayıcı	2
1.2	Elle Derlemenin Sınırları	3
1.3	Make: İlk Çözüm ve Sınırları	3
2	CMake Nedir ve Nasıl Çalışır?	4
2.1	İki Aşama: Configure/Generate ve Build	4
2.2	Generator Kavramı	4
3	Kurulum ve İlk Proje	5
3.1	CMake Kurulumu	5
3.2	En Küçük C++ Projesi	5
3.3	En Küçük C Projesi	6
3.4	Faydalı Komut Satırı Bayrakları	6
4	CMake Dilinin Temelleri	8
4.1	Değişkenler ve set/unset	8
4.2	Değişken Kapsamları (Scope)	8
4.3	Cache Değişkenleri	9
4.4	Liste İşlemleri	9
4.5	String İşlemleri	9
4.6	Koşullar	10
4.7	Döngüler	10
5	Hedefler (Targets): Modern CMake'in Kalbi	11
5.1	add_executable ve add_library	11
5.2	PUBLIC, PRIVATE, INTERFACE — Yayılım (Propagation)	11
5.3	target_include_directories	12
5.4	target_link_libraries	12
5.5	target_compile_features — Standart Belirlemenin Doğru Yolu	12
5.6	target_compile_definitions ve target_compile_options	12
5.7	Bir Hedefin Tüm Özelliklerini Bir Arada	13
6	Kütüphane Türleri	14
6.1	Statik Kütüphane (STATIC): Adım Adım	14
6.2	Dinamik (Paylaşımlı) Kütüphane (SHARED): Adım Adım	16
6.3	Header-Only Kütüphane (INTERFACE)	17
6.4	ALIAS Hedefleri — Namespace'li İsimler	17
7	Çok Dizinli Proje Düzeni	18
7.1	Önerilen Klasör Yapısı	18
7.2	Kök CMakeLists.txt	18
7.3	Kütüphane Alt Dizini	19
8	Harici Bağımlılıklar: find_package	20
8.1	İki Mod: Config ve Module	20
8.2	Örnek: Threads (pthread)	20
8.3	Örnek: fmt Kütüphanesi (Config mod)	20
8.4	COMPONENTS ve Opsiyonel Kullanım	21
8.5	Elle Kurulan Kütüphaneler: CMake Nerede Arar?	21

9	Fonksiyonlar ve Makrolar	24
9.1	function vs macro	24
9.2	Bir Fonksiyonun Anatomisi: Parametreler ve ARGN	24
9.3	Neden Fonksiyon? Önce/Sonra	25
9.4	Gerçek Proje: Fonksiyonu Ayrı Bir Modüle Koymak	25
9.5	İsimli Argümanlar: cmake_parse_arguments	27
10	Generator Expressions	28
10.1	Neden Gerekir?	29
10.2	Derleyiciye Göre Bayrak	29
10.3	Derleme Tipine Göre Tanım	29
10.4	Sık Kullanılan Genex Kalıpları	29
10.5	BUILD_INTERFACE / INSTALL_INTERFACE	29
11	Derleme Tipleri, Uyarılar ve Sanitizer'lar	30
11.1	CMAKE_BUILD_TYPE	30
11.2	Uyarıları Merkezi Açmak	31
11.3	Sanitizer'lar (ASan, UBSan)	31
12	FetchContent: Bağımlılığı Projeye Çekmek	33
12.1	FetchContent + find_package Hibrit (3.24+)	33
13	Test: CTest ile	34
13.1	Temel Test Kaydı	34
13.2	GoogleTest ile Entegrasyon	35
14	Kurulum (install) ve Paketleme (CPack)	36
14.1	install Komutu	36
14.2	CPack ile Dağıtım Paketi	37
15	Kendi Config Paketini Yazmak	37
16	CMakePresets.json — Yapılandırmayı Sabitlemek	38
17	Toolchain Dosyaları ve Çapraz Derleme	40
18	Tam Örnek: Çok Modüllü C++ Projesi	41
18.1	Dosya Yapısı	41
18.2	Kütüphane Kaynağı	41
18.3	Tüm CMakeLists.txt Dosyaları	42
18.4	Derleme ve Çalıştırma	43
19	Tam Örnek: C Projesi (Kütüphane + Uygulama)	43
19.1	Dosya Yapısı	43
19.2	Kaynaklar	43
19.3	CMakeLists.txt Dosyaları	44
20	En İyi Uygulamalar	45
21	Sık Karşılaşılan Hatalar ve Çözümleri	46

22 Hızlı Başvuru (Cheat Sheet)	46
22.1 En Sık Komutlar	46
22.2 En Sık Terminal Komutları	47
22.3 Önemli Yerleşik Değişkenler	47

KISIM 1

Temeller — Derleme, make ve CMake

1. Bir Derleme Sistemi Neden Gerekir?

CMake'i öğrenmenin ilk adımı, onun *hangi soruna* çözüm getirdiğini görmektir. Yazdığımız bir C ya da C++ programı olduğu gibi çalışmaz; kaynak kod (.c ve .cpp dosyaları) önce bir dizi araçtan geçip makine koduna dönüştürülür. Tek bir küçük dosyada bu adımları elle yürütmek kolaydır. Ne var ki proje büyüdükçe — onlarca dosya, farklı kütüphaneler, birden çok işletim sistemi — bu işi elle takip etmek olanaksız hale gelir. Derleme sistemleri de tam olarak bu karmaşıklığı sizin yerinize yönetmek için vardır.

1.1 Derleme Zinciri: Önilemci, Derleyici, Bağlayıcı

Bir kaynak dosyası çalıştırılabilir bir programa dönüşürken üç temel aşamadan geçer. Bu aşamaları tanımak, ileride karşılaşacağınız hataları — özellikle “undefined reference” türünden bağlayıcı hatalarını — çözmenin anahtarıdır.

Aşama	Ne yapar?	Girdi → Çıktı
Önilemci	#include, #define, #ifdef gibi direktifleri işler; başlık dosyalarını metin olarak yerleştirir.	.c/.cpp → genişletilmiş kaynak
Derleyici	Genişletilmiş kaynağı makine koduna çevirir; her dosya için bir nesne (object) dosyası üretir.	.cpp → .o
Bağlayıcı (linker)	Tüm nesne dosyalarını ve kütüphaneleri birleştirir; sembol referanslarını çözer; tek bir çalıştırılabilir üretir.	.o + kütüphaneler → a.out

Örneğin main.cpp dosyasını elle derlemek şöyle görünür:

```

1 # All in one step (preprocess + compile + link together)
2 g++ -std=c++17 main.cpp -o program
3
4 # Step by step: first the object file, then the linking stage
5 g++ -std=c++17 -c main.cpp -o main.o      # compile only: .cpp -> .o
6 g++ main.o -o program                    # link: .o -> executable
7
```

```

8 # Inspect the produced binary and its symbols
9 file program          # ELF 64-bit executable, dynamically linked, ...
10 nm main.o | head      # list symbols defined/referenced in the object file

```

1.2 Elle Derlemenin Sınırları

Tek dosya için yukarıdaki komut yeterli. Ama gerçek bir proje düşünün:

```

1 # Compile every translation unit separately (-I adds a header search path)
2 g++ -std=c++17 -Iinclude -c src/main.cpp -o build/main.o
3 g++ -std=c++17 -Iinclude -c src/matrix.cpp -o build/matrix.o
4 g++ -std=c++17 -Iinclude -c src/vector.cpp -o build/vector.o
5 g++ -std=c++17 -Iinclude -c src/parser.cpp -o build/parser.o
6
7 # Link all object files together with the required libraries
8 # (-lpthread = POSIX threads, -lm = C math library)
9 g++ build/main.o build/matrix.o build/vector.o build/parser.o \
10     -lpthread -lm -o program

```

Bu yaklaşımın sorunları:

- **Tekrar derleme:** Sadece matrix.cpp değişse bile, elle her şeyi yeniden derlersiniz. Hangi dosyanın değiştiğini takip etmek gerekir.
- **Taşınabilirlik yok:** Windows'ta g++ yerine cl.exe (MSVC), farklı bayraklar gerekir. Komutlarınız tek bir platforma bağlı kalır.
- **Bağımlılık yönetimi:** Harici bir kütüphane (ör. Boost, SQLite) kullanacaksanız onun yolunu, bayraklarını elle bulmak zorundasınız.
- **Ölçeklenmez:** 200 dosyalık bir projede bu komutları yazmak ve güncellemek imkansızdır.

1.3 Make: İlk Çözüm ve Sınırları

Bu soruna getirilen ilk büyük çözüm make oldu. Makefile adında bir dosyaya “hangi çıktı, hangi girdilerden ve nasıl üretilir” kurallarını yazarsınız; make da yalnızca *değişen* dosyaları yeniden derler.

```

1 # Variables reused throughout the recipes
2 CXX      = g++
3 CXXFLAGS = -std=c++17 -Iinclude
4 OBJ      = build/main.o build/matrix.o build/vector.o
5
6 # The final target depends on all object files; relink only when they change
7 program: $(OBJ)
8     $(CXX) $(OBJ) -o program
9
10 # Pattern rule: how to build any build/X.o from src/X.cpp
11 # $< = the first prerequisite (the .cpp), $@ = the target (the .o)
12 build/%.o: src/%.cpp
13     $(CXX) $(CXXFLAGS) -c $< -o $@

```

make güçlü bir araçtır, ama hâlâ platforma bağımlıdır: Windows’un yerel derleyicisi make kullanmaz, Visual Studio kendi proje biçimini (.sln/.vcxproj) bekler. Üstelik bağımlılıkları bulmayı, sürümleri denetlemeyi, kurulum ve paketlemeyi de elle çözmeniz gerekir. CMake işte bu boşluğu doldurmak için geliştirildi.

2. CMake Nedir ve Nasıl Çalışır?

CMake bir *derleyici değildir*; kodu kendisi derlemez. O bir **meta derleme sistemidir** (build system generator): siz platformdan bağımsız tek bir tarif (CMakeLists.txt) yazarsınız, CMake de bunu içinde bulunduğunuz platformun *yerel* derleme dosyalarına dönüştürür — Linux'ta Makefile ya da Ninja dosyalarına, Windows'ta Visual Studio projelerine, macOS'ta Xcode projelerine. Böylece tek bir tarif her yerde çalışır.

Bilgi

Özet zihinsel model: **Sen** CMakeLists.txt yazarsın → **CMake** onu okuyup build/ klasörüne Ninja/Make/VS dosyaları *üretir* (configure + generate) → **Ninja/Make** o dosyalarla g++/clang/cl'i çağırıp kodu *derler* (build).

2.1 İki Aşama: Configure/Generate ve Build

CMake ile çalışmak her zaman iki mantıksal aşamadan oluşur:

Aşama	Açıklama
Configure & Generate	CMake CMakeLists.txt dosyalarını okur, derleyiciyi tespit eder, seçenekleri değerlendirir ve build/ içine yerel derleme dosyalarını üretir. Komut: <code>cmake -S . -B build</code>
Build	Üretilen dosyalar kullanılarak asıl derleme yapılır (nesne dosyaları, kütüphaneler, çalıştırılabilirler). Komut: <code>cmake --build build</code>

2.2 Generator Kavramı

CMake'in hangi yerel derleme sistemini üreteceğini “generator” (üretici) belirler. En yaygın kullanılanlar şunlardır:

Generator	Açıklama
Unix Makefiles	Klasik make dosyaları üretir (Linux/macOS varsayılanı).
Ninja	Çok hızlı, paralel derleme için ideal. Önerilir. <code>-G Ninja</code>
Visual Studio 17 2022	Windows'ta .sln çözümü üretir.
Xcode	macOS'ta Xcode projesi üretir.

```

1 cmake -S . -B build -G "Ninja"           # use Ninja (recommended)
2 cmake -S . -B build -G "Unix Makefiles"  # use classic Make
3 cmake -S . -B build -G "Visual Studio 17 2022" # generate a VS solution
4 cmake --help                             # list all available generators

```

İpucu

Ninja'yı öneririz: hem hızlıdır hem de tüm platformlarda aynı davranır. Kurulum: Linux `sudo pacman -S ninja / apt install ninja-build`, macOS `brew install ninja`, Windows `winget install Ninja-build.Ninja`.

3. Kurulum ve İlk Proje

3.1 CMake Kurulumu

```

1  # Linux (Arch)
2  sudo pacman -S cmake ninja
3
4  # Linux (Debian/Ubuntu)
5  sudo apt install cmake ninja-build
6
7  # macOS
8  brew install cmake ninja
9
10 # Windows
11 winget install Kitware.CMake Ninja-build.Ninja
12
13 # Verify the installation (CMake 3.20 or newer is assumed in this guide)
14 cmake --version
15 ninja --version

```

3.2 En Küçük C++ Projesi

Bir klasör açalım ve iki dosya koyalım.

main.cpp

```

1  #include <iostream>
2
3  int main() {
4      // std::cout is the standard output stream; std::endl flushes the buffer.
5      std::cout << "Hello, CMake!" << std::endl;
6      return 0; // returning 0 from main() signals success to the OS
7  }

```

CMakeLists.txt

```

1  # Minimum required CMake version. Make this the first line of every project.
2  # It also decides which policy defaults ("modern behavior") are enabled.
3  cmake_minimum_required(VERSION 3.20)
4
5  # Project name, version and the languages used. CXX = C++, C = the C language.
6  # This call sets variables like PROJECT_NAME and PROJECT_VERSION for you.
7  project(HelloCMake VERSION 1.0 LANGUAGES CXX)
8
9  # Select the C++ standard used to compile all targets defined afterwards.
10 set(CMAKE_CXX_STANDARD 17)           # compile as C++17
11 set(CMAKE_CXX_STANDARD_REQUIRED ON)  # fail instead of silently falling back
12 set(CMAKE_CXX_EXTENSIONS OFF)        # use -std=c++17, not the GNU -std=gnu++17
13
14 # Create an executable target: target name + its source files.
15 add_executable(hello main.cpp)

```

Şimdi derleyip çalıştıralım:

```

1  cmake -S . -B build -G Ninja      # 1) configure + generate
2  cmake --build build               # 2) build (compiles main.cpp, links hello)
3  ./build/hello                     # 3) run -> Hello, CMake!

```


Not

-S . kaynak dizini (CMakeLists.txt'nin bulunduğu yer), -B build ise derleme çıktıların yazılacağı dizindir. Bu ayırım **out-of-source build** denen iyi uygulamadır: kaynağınız temiz kalır, build/ klasörünü silerek her şeyi sıfırlayabilirsiniz.

3.3 En Küçük C Projesi

Aynı yapı C için de neredeyse birebir aynıdır; tek fark LANGUAGES C ve C standardıdır.

main.c

```
1 #include <stdio.h>
2
3 int main(void) {
4     /* printf writes formatted text to standard output; \n adds a newline. */
5     printf("Hello, CMake (C)!\n");
6     return 0;    /* 0 = success */
7 }
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.20)
2 project(HelloC VERSION 1.0 LANGUAGES C)    # note: LANGUAGES C, not CXX
3
4 set(CMAKE_C_STANDARD 11)                  # compile as C11
5 set(CMAKE_C_STANDARD_REQUIRED ON)         # do not silently fall back to an older C
6 set(CMAKE_C_EXTENSIONS OFF)              # use -std=c11 instead of -std=gnu11
7
8 add_executable(hello_c main.c)
```

```
1 cmake -S . -B build -G Ninja
2 cmake --build build
3 ./build/hello_c    # -> Hello, CMake (C)!
```

Dikkat

build/ klasörünü **asla** sürüm kontrolüne (git) eklemeyin. .gitignore dosyanıza build/ satırını ekleyin. Bu klasör tümüyle üretilir ve makineden makineye taşınmaz.

3.4 Faydalı Komut Satırı Bayrakları

Bayrak	Ne yapar
-S <dir>	Kaynak dizini (CMakeLists.txt konumu).
-B <dir>	Derleme (binary) dizini.
-G <gen>	Generator seç (Ninja, "Unix Makefiles"...).
-D VAR=DEG	Bir cache değişkenine değer ata.
-build <dir>	Yapılandırılmış projeyi derle.
-build <dir> -j 8	8 iş parçacığıyla paralel derle.
-build <dir> --target X	Sadece X hedefini derle.
-fresh	Cache'i yok sayıp baştan yapılandır (3.24+).
-install <dir>	Projeyi kur (install aşaması).

KISIM 2

CMake Dili ve Hedefler

4. CMake Dilinin Temelleri

CMakeLists.txt aslında küçük bir betik dilidir. Dosyadaki her satır bir *komut çağrısıdır*: `komut (argüman1 argüman2 ...)`. Bu dilin en şaşırtıcı yanı şudur: **her şey birer metindir (string)**. Sayılar, listeler, mantıksal değerler... hepsi arka planda metin olarak saklanır.

4.1 Değişkenler ve set/unset

```

1  # Define a variable (its value is ALWAYS stored as text)
2  set(MY_NAME "Tolga")
3  set(MY_VERSION 42)
4
5  # Use a variable: ${...} "dereferences" it (substitutes its value)
6  message(STATUS "Name: ${MY_NAME}, Version: ${MY_VERSION}")
7
8  # A list is really a single string with ; as the element separator
9  set(SOURCES main.cpp util.cpp parser.cpp)    # stored as "main.cpp;util.cpp;parser.cpp"
10
11 # An undefined variable expands to an empty string (no error is raised)
12 message(STATUS "Undefined -> '${DOES_NOT_EXIST}')"    # -> Undefined -> ''
13
14 # Remove a variable
15 unset(MY_NAME)

```

```

1  -- Name: Tolga, Version: 42
2  -- Undefined -> ''

```

Not

`message(STATUS ...)` yapılandırma sırasında ekrana bilgi yazar — hata ayıklamanın en pratik yolu. Diğer seviyeler: `WARNING`, `FATAL_ERROR` (yapılandırmayı durdurur), `DEBUG`.

4.2 Değişken Kapsamları (Scope)

CMake’te bir değişken, varsayılan olarak tanımlandığı *dizin* ve *fonksiyon* kapsamında yaşar. Bir alt dizine geçildiğinde (`add_subdirectory`) değişkenlerin bir kopyası o alt dizine aktarılır. Bir değişikliği üst kapsama geri taşımak isterseniz `PARENT_SCOPE` anahtarını kullanırsınız.

```

1  function(demo_scope)

```

```

2     set(X "local")                # visible only inside this function
3     set(Y "outer" PARENT_SCOPE)   # written into the CALLER's scope
4 endfunction()
5
6 demo_scope()
7 message(STATUS "X='${X}' Y='${Y}'") # -> X='' Y='outer'

```

4.3 Cache Değişkenleri

Sıradan değişkenler her yapılandırmada sıfırdan oluşturulur. **Cache (önbellek) değişkenleri** ise build/CMakeCache.txt dosyasında kalıcı olarak saklanır ve komut satırından -D ile değiştirilebilir. Kullanıcının dışarıdan ayarlamasını istediğiniz seçenekler için bu tür değişkenleri kullanın.

```

1 # A cache variable needs a type (STRING/BOOL/PATH/FILEPATH) and a description
2 set(MY_LIB_PATH "/usr/local" CACHE PATH "Library root directory")
3
4 # option() is a convenient shortcut for a boolean cache variable
5 option(BUILD_TESTS "Build the unit tests" ON)
6
7 # Using it
8 if(BUILD_TESTS)
9     message(STATUS "Tests will be built")
10 endif()

```

```

1 # Override cache values from the command line with -D:
2 cmake -S . -B build -D BUILD_TESTS=OFF -D MY_LIB_PATH=/opt/mylib
3
4 # Inspect or edit all cache variables interactively:
5 cmake -L build          # list cache variables and their values
6 ccmake build            # curses-based cache editor (if installed)

```

Bilgi

option(X "... " ON) aslında set(X ON CACHE BOOL "...") kısaltmasıdır. Mantıksal değerler için ON/OFF, TRUE/FALSE, 1/0, YES/NO kullanılabilir.

4.4 Liste İşlemleri

```

1 set(FILES a.cpp b.cpp c.cpp)
2
3 list(APPEND FILES d.cpp)          # append -> a;b;c;d
4 list(PREPEND FILES z.cpp)         # prepend -> z;a;b;c;d
5 list(REMOVE_ITEM FILES b.cpp)     # remove an element by value -> z;a;c;d
6 list(LENGTH FILES count)          # element count -> count = 4
7 list(GET FILES 0 first)           # index access -> first = z.cpp
8 list(FIND FILES c.cpp idx)        # index of an element -> idx = 2 (-1 if absent)
9 list(SORT FILES)                  # sort in place -> a;c;d;z
10 list(REVERSE FILES)               # reverse in place -> z;d;c;a
11 list(REMOVE_DUPLICATES FILES)     # drop repeated elements, keep first
12 list(JOIN FILES " " joined)       # list -> string: "z.cpp, d.cpp, c.cpp, a.cpp"

```

4.5 String İşlemleri

```

1 string(TOUPPER "hello" upper)     # upper = "HELLO"
2 string(TOLOWER "CMake" lower)     # lower = "cmake"

```

```

3 string(LENGTH "cmake" len)           # len = 5
4 string(REPLACE "a" "@" out "cmake")  # out = "cm@ke"
5 string(SUBSTRING "cmake" 0 2 head)    # head = "cm" (start, length)
6 string(FIND "hello.cpp" ".cpp" pos)   # pos = 5 (index, -1 if absent)
7 string(REGEX MATCH "[0-9]+" num "abc123") # num = "123" (first match)
8 string(REGEX REPLACE "[0-9]" "#" masked "a1b2") # masked = "a#b#"

```

4.6 Koşullar

```

1 if(WIN32)
2     message(STATUS "We are on Windows")
3 elseif(APPLE)
4     message(STATUS "We are on macOS")
5 elseif(UNIX)
6     message(STATUS "We are on Linux/Unix")
7 endif()
8
9 # Comparison operators
10 if(CMAKE_CXX_COMPILER_VERSION VERSION_GREATER_EQUAL "11.0") # version compare
11 endif()
12 if(name STREQUAL "tolga") # string equality
13 endif()
14 if(x EQUAL 42) # numeric equality
15 endif()
16 if(DEFINED MY_VAR) # is the variable defined?
17 endif()
18 if(EXISTS "${CMAKE_SOURCE_DIR}/config.h") # does the file/directory exist?
19 endif()
20 if(NOT BUILD_TESTS AND USE_GUI) # boolean operators: NOT / AND / OR
21 endif()
22 if("release" IN_LIST ALLOWED_TYPES) # membership test on a list variable
23 endif()

```

4.7 Döngüler

```

1 # Iterate over the elements of a list
2 set(MODULES core net ui)
3 foreach(mod IN_LISTS MODULES)
4     message(STATUS "Module: ${mod}")
5 endforeach()
6
7 # Iterate over several items directly (IN ITEMS)
8 foreach(name IN ITEMS alice bob carol)
9     message(STATUS "User: ${name}")
10 endforeach()
11
12 # Numeric range (inclusive): 1,2,3,4,5
13 foreach(i RANGE 1 5)
14     message(STATUS "i = ${i}")
15 endforeach()
16
17 # while loop with arithmetic via math(EXPR ...)
18 set(n 3)
19 while(n GREATER 0)
20     message(STATUS "n = ${n}")
21     math(EXPR n "${n} - 1") # evaluate an integer expression
22 endwhile()

```

5. Hedefler (Targets): Modern CMake'in Kalbi

Modern CMake'in en temel kavramı **hedef** (target). Hedef, derlenecek bir şeydir: bir çalıştırılabilir program ya da bir kütüphane. Eski CMake, `include_directories()` veya `add_definitions()` gibi *global* komutlara dayanırdı; bunlar tüm projeyi birden etkilediği için büyük projelerde işler kolayca içinden çıkılmaz hale gelirdi. Modern yaklaşım ise her ayar **ilgili hedefe bağlar**: “bu hedef şu başlıklara, şu bayraklara ve şu kütüphanelere ihtiyaç duyar” dersiniz.

Bilgi

Altın kural: **Global komutlardan kaçın** (`include_directories`, `link_libraries`, `add_definitions`). Bunun yerine `target_*` komutlarını kullan. Her ayar, ait olduğu hedefe bağlansın.

5.1 `add_executable` ve `add_library`

```

1 # Executable target
2 add_executable(app main.cpp app.cpp)
3
4 # Static library (.a / .lib) -- compiled into and embedded in the consumer
5 add_library(math STATIC matrix.cpp vector.cpp)
6
7 # Shared library (.so / .dll / .dylib) -- loaded at run time
8 add_library(math_shared SHARED matrix.cpp vector.cpp)
9
10 # If you omit the type, the BUILD_SHARED_LIBS variable decides
11 # (defaults to STATIC when BUILD_SHARED_LIBS is unset)
12 add_library(math_auto matrix.cpp vector.cpp)

```

5.2 PUBLIC, PRIVATE, INTERFACE — Yayılım (Propagation)

`target_*` komutlarındaki bu üç anahtar sözcük, bir ayarın *kimi* etkileyeceğini belirler. Bu üçünü kavramak, modern CMake'i kavramanın ta kendisidir.

Anahtar

Anlamı

PRIVATE	Sadece <i>bu hedefin kendisi</i> kullanır. Bu hedefe bağlanan başkaları etkilenmez. (Uygulama detayı.)
INTERFACE	Bu hedefin kendisi kullanmaz ama <i>ona bağlananlar</i> kullanır. (Sadece başlıklı — header-only — kütüphaneler için tipik.)
PUBLIC	Hem bu hedef hem de <i>ona bağlananlar</i> kullanır. (PRIVATE + INTERFACE.)

Sezgi

Bir kütüphane, başlığında (`.h`) bir tipi `#include` ediyorsa, o başlığı kullanan *herkes* de o `include` yoluna ihtiyaç duyar → **PUBLIC**. Sadece `.cpp` içinde, gizli bir şekilde kullanıyorsa → **PRIVATE**.

5.3 `target_include_directories`

Bir hedefin başlık dosyalarını nerede arayacağını söyler.

```
1 add_library(math STATIC src/matrix.cpp src/vector.cpp)
2
3 target_include_directories(math
4     PUBLIC
5     ${CMAKE_CURRENT_SOURCE_DIR}/include    # consumers of math.h see this too
6     PRIVATE
7     ${CMAKE_CURRENT_SOURCE_DIR}/src        # only math's own source files
8 )
```

5.4 target_link_libraries

Bir hedefi başka bir hedefe ya da kütüphaneye bağlar. Bu komutun asıl gücü şuradadır: bağlandığınız hedefin PUBLIC ve INTERFACE özellikleri — include yolları, tanımlar, bayraklar — *kendiliğinden* sizin hedefinize de geçer.

```
1 add_library(math STATIC src/matrix.cpp)
2 target_include_directories(math PUBLIC include)
3
4 add_executable(app main.cpp)
5 # app AUTOMATICALLY inherits math's PUBLIC include path!
6 target_link_libraries(app PRIVATE math)
```

İpucu

`target_link_libraries(app PRIVATE math)` yazdığınızda `math/include` yolunu ayrıca app'a eklemeniz gerekmez. Bu “kullanım gereksinimlerinin” (usage requirements) otomatik yayılımı, modern CMake'in en güçlü özelliğidir.

5.5 target_compile_features — Standart Belirlemenin Doğru Yolu

`set(CMAKE_CXX_STANDARD 17)` tüm projeyi birden etkiler. Standardı hedef hedef belirlemek ve bu gereksinimi bağlanan hedeflere de yaymak için `target_compile_features` tercih edilir.

```
1 add_library(math STATIC src/matrix.cpp)
2
3 # This library requires C++17; anything that links it also gets at least C++17.
4 target_compile_features(math PUBLIC cxx_std_17)
5
6 # For C:
7 # target_compile_features(mylib PUBLIC c_std_11)
```

5.6 target_compile_definitions ve target_compile_options

```
1 add_executable(app main.cpp)
2
3 # Preprocessor macros (equivalent to -DNAME=VALUE on the command line)
4 target_compile_definitions(app PRIVATE
5     APP_VERSION="1.2.0"        # in C++: #define APP_VERSION "1.2.0"
6     ENABLE_LOGGING             # value-less macro, tested with #ifdef ENABLE_LOGGING
7 )
8
9 # Compiler flags (use with care -- these are not portable!)
10 target_compile_options(app PRIVATE
11     -Wall -Wextra -Wpedantic
12 )
```

Dikkat

Derleyici bayrakları taşınabilir değildir: -Wall GCC/Clang'te çalışır ama MSVC'de /W4 gerekir. Bu tür bayrakları generator expression (Kısım 3) ile derleyiciye göre koşullamak gerekir.

5.7 Bir Hedefin Tüm Özelliklerini Bir Arada

```

1  add_library(graphics STATIC
2      src/renderer.cpp
3      src/texture.cpp
4      src/shader.cpp
5  )
6
7  target_include_directories(graphics
8      PUBLIC include           # for consumers that include graphics.h
9      PRIVATE src              # internal implementation details
10 )
11
12 target_compile_features(graphics PUBLIC cxx_std_17)
13
14 target_compile_definitions(graphics
15     PUBLIC  GRAPHICS_VERSION="2.0"
16     PRIVATE GRAPHICS_INTERNAL_BUILD
17 )
18
19 target_link_libraries(graphics
20     PUBLIC  math              # if math types appear in graphics' public headers
21     PRIVATE pthread          # used only inside the implementation
22 )

```

MINİ PROJE • Yayılımı canlı görmek: uygulama hiçbir yol/makro yazmadan derleniyor

Aşağıdaki projede app, ne başlık yolu ne de bir makro tanımlar; hepsini greet kütüphanesinin PUBLIC özelliklerinden *otomatik* devralır. İşte modern CMake'in tüm mantığı budur.

```

1  targets-demo/
2  |-- CMakeLists.txt
3  |-- include/greet.h
4  |-- src/greet.cpp
5  '-- app/main.cpp

```

include/greet.h

```

1  #pragma once
2  #include <string>
3  namespace greet { std::string hello(); }

```

src/greet.cpp

```

1  #include "greet.h"
2  namespace greet { std::string hello() { return "Merhaba, hedefler!"; } }

```

app/main.cpp


```

1 #include <iostream>
2 #include "greet.h" // include yolu greet'ten otomatik geldi
3 int main() {
4     std::cout << greet::hello() << "\n";
5     std::cout << "Surum: " << GREET_VERSION << "\n"; // makro da greet'ten geldi
6 }

```

CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.20)
2 project(TargetsDemo LANGUAGES CXX)
3
4 add_library(greet STATIC src/greet.cpp)
5 target_include_directories(greet PUBLIC include) # app da gorecek
6 target_compile_definitions(greet PUBLIC GREET_VERSION="1.0") # app da gorecek
7 target_compile_features(greet PUBLIC cxx_std_17) # app da gorecek
8
9 add_executable(app app/main.cpp)
10 target_link_libraries(app PRIVATE greet) # ustteki 3 PUBLIC ozellik app'e gecer
11 # DIKKAT: app icin ne include yolu ne de makro yazdik -- gerek kalmadi!

```

```

1 cmake -S . -B build -G Ninja && cmake --build build
2 ./build/app
3 # Merhaba, hedefler!
4 # Surum: 1.0

```

6. Kütüphane Türleri

Tür	Açıklama
STATIC	.a/.lib — kod, kullanan programa gömülür. Bağımlılık dağıtmak kolaydır.
SHARED	.so/.dll/.dylib — çalışma anında yüklenir. Bellek paylaşımı, ayrı güncelleme.
MODULE	Çalışma anında dlopen ile açılan eklenti (plugin). Bağlanmaz.
INTERFACE	Derlenecek kaynağı yok; sadece kullanım gereksinimleri (header-only kütüphaneler).
OBJECT	Nesne dosyaları (.o) üretir, arşivlenmez; başka hedeflere gömülmek için.

Bu türlerden en sık kullandığımız ikisi **statik** ve **paylaşımlı (dinamik)** kütüphanelerdir. İkisinin de kaynak kodu ve `add_library` çağırısı neredeyse birebir aynıdır; aradaki fark, kütüphanenin makine kodunun *ne zaman* ve *nasıl* programa dahil olduğudur. Aşağıda ikisini de sıfırdan, çalışan birer projeye kuruyoruz. Her iki proje de *aynı* kaynak dosyalarını kullanır; yalnızca `CMakeLists.txt` içindeki tek kelime değişir.

6.1 Statik Kütüphane (STATIC): Adım Adım

Statik kütüphane derlendiğinde bir `.a` dosyası (Windows'ta `.lib`) üretir. Bu dosyadaki makine kodu, kütüphaneyi kullanan programa *bağlama anında* kopyalanır. Sonuçta ortaya tek ve kendi kendine yeten bir çalıştırılabilir çıkar: dağıtırken yanında ayrıca kütüphane taşımanız gerekmez. Bedeli ise, ikili dosyanın biraz büyümesi ve kütüphaneyi güncellediğinizde programı yeniden bağlamanız gerekesidir.

MINİ PROJE • Statik geo kütüphanesi + onu kullanan uygulama

Şu klasör yapısını kuralım:

```
1 geo-static/
2 |-- CMakeLists.txt
3 |-- include/geo.h           # kutuphanenin genel arayuzu (başlık)
4 |-- src/geo.cpp             # kutuphanenin gövdesi (kaynak)
5 '-- app/main.cpp           # kutuphaneyi kullanan uygulama
```

include/geo.h

```
1 #pragma once
2 namespace geo {
3     double circle_area(double r);           // daire alanı
4     double rect_area(double w, double h);   // dikdörtgen alanı
5 }
```

src/geo.cpp

```
1 #include "geo.h"
2 namespace geo {
3     static const double PI = 3.14159265358979;
4     double circle_area(double r)           { return PI * r * r; }
5     double rect_area(double w, double h) { return w * h; }
6 }
```

app/main.cpp

```
1 #include <iostream>
2 #include "geo.h"
3
4 int main() {
5     std::cout << "Daire (r=2)      : " << geo::circle_area(2) << "\n";
6     std::cout << "Dikdörtgen 3x4 : " << geo::rect_area(3, 4) << "\n";
7 }
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.20)
2 project(GeoStatic VERSION 1.0 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 # 1) STATIC kutuphane: derlenince libgeo.a (arsiv) üretir
8 add_library(geo STATIC src/geo.cpp)
9
10 # 2) geo.h'yi kullanan herkes include/ dizinini görebilsin (PUBLIC)
11 target_include_directories(geo PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include)
12
13 # 3) Uygulamayı tanımla ve kutuphaneye bağla
14 add_executable(app app/main.cpp)
15 target_link_libraries(app PRIVATE geo) # geo'nun kodu app'in içine gömülür
```

```
1 cmake -S . -B build -G Ninja
2 cmake --build build
```

```

3 ./build/app
4 #   Daire (r=2)      : 12.5664
5 #   Dikdortgen 3x4   : 12
6
7 ls build/libgeo.a      # <- uretilen statik kutuphane (arsiv dosyasi)
8 ldd build/app | grep geo # <- CIKTI YOK: geo'nun kodu app'e gomuldu,
9                          #   calisma aninda ayri bir dosyaya ihtiyac yok

```

Bilgi

ldd komutunun çıktısında geo'yu *görmezsiniz*; çünkü statik kütüphanede kod, çalıştırılabilirin içine kopyalanmıştır. ./build/app dosyasını başka bir makineye kopyalasanız, libgeo.a olmadan da çalışır.

6.2 Dinamik (Paylaşımlı) Kütüphane (SHARED): Adım Adım

Paylaşımlı kütüphane; Linux'ta .so, Windows'ta .dll, macOS'ta .dylib dosyası üretir. Bu kez kod programa kopyalanmaz, ayrı bir dosyada kalır ve *çalışma anında* yüklenir. Böylece aynı kütüphaneyi birçok program paylaşabilir ve kütüphaneyi, programları yeniden derlemeden güncelleyebilirsiniz. Karşılığında bir sorumluluk gelir: program çalışırken bu .so dosyasını *bulabilmelidir*; aksi halde “cannot open shared object file” hatası alırsınız.

MINİ PROJE • Aynı geo projesi, bu kez paylaşımlı kütüphaneyle

Kaynak dosyaların (geo.h, geo.cpp, main.cpp) hepsi **birebir aynı** kalır. Değişen tek şey CMakeLists.txt:

CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.20)
2 project(GeoShared VERSION 1.0 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 # TEK FARK: STATIC yerine SHARED -> libgeo.so üretir
8 add_library(geo SHARED src/geo.cpp)
9 target_include_directories(geo PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include)
10
11 # İyi alışkanlık: kütüphaneye surum ver -> libgeo.so -> libgeo.so.1 -> .so.1.0
12 set_target_properties(geo PROPERTIES VERSION 1.0 SOVERSION 1)
13
14 add_executable(app app/main.cpp)
15 target_link_libraries(app PRIVATE geo) # geo bu sefer app'e GOMULMEZ, disarida kalir

```

```

1 cmake -S . -B build -G Ninja
2 cmake --build build
3 ./build/app
4 #   Daire (r=2)      : 12.5664
5 #   Dikdortgen 3x4   : 12
6
7 ls build/libgeo.so*      # libgeo.so libgeo.so.1 libgeo.so.1.0
8 ldd build/app | grep geo # libgeo.so.1 => ../build/libgeo.so.1
9                          # <- calisma aninda GEREKLI: kod ayri dosyada

```

İpucu

Yukarıda `./build/app` sorunsuz çalıştı; çünkü CMake, derleme ağacındaki kütüphanenin yolunu çalıştırılabilirin içine (RPATH olarak) otomatik gömer. Kütüphaneyi `install` ile başka bir yere kurup taşırsanız, çalışma anında bulunması için ya kurulum RPATH'i ayarlanmalı ya da `LD_LIBRARY_PATH` kütüphanenin dizinini içermelidir.

Hangisini seçmeli?

Statik: dağıtımı kolaydır (tek dosya), çalışma anında bağımlılık yoktur; ama ikili büyür ve güncelleme için yeniden bağlama gerekir. **Paylaşımlı:** bellek ve disk paylaşımı sağlar, ayrı güncellenebilir; ama `.so` çalışma anında bulunmalıdır. Kararsızsanız statikle başlayın. `add_library`'de tür yazmazsanız seçimi `BUILD_SHARED_LIBS` değişkeni yapar: `-D BUILD_SHARED_LIBS=ON` ile projedeki tüm kütüphaneleri tek hamlede paylaşımlıya çevirebilirsiniz.

6.3 Header-Only Kütüphane (INTERFACE)

Yalnızca başlık dosyalarından oluşan bir kütüphaneyi (birçok modern C++ kütüphanesi böyledir) INTERFACE olarak tanımlarsınız; ortada derlenecek bir `.cpp` yoktur.

```

1 # Only header files (.hpp) under include/ exist -- nothing to compile
2 add_library(minimatrix INTERFACE)
3
4 target_include_directories(minimatrix INTERFACE
5     ${CMAKE_CURRENT_SOURCE_DIR}/include
6 )
7 target_compile_features(minimatrix INTERFACE cxx_std_17)
8
9 # Usage:
10 add_executable(demo demo.cpp)
11 target_link_libraries(demo PRIVATE minimatrix) # gets the include path automatically

```

6.4 ALIAS Hedefleri — Namespace'li İsimler

Kütüphanelere `Ad::Alt` biçiminde takma ad vermek yerleşmiş, profesyonel bir alışkanlıktır; böylece kütüphaneniz `find_package` ile gelen hedeflerle aynı görünümü kazanır.

```

1 add_library(math STATIC src/matrix.cpp)
2 add_library(MiniMat::math ALIAS math) # namespaced alias name
3
4 # Now it can be used like this (as if it came from find_package):
5 target_link_libraries(app PRIVATE MiniMat::math)

```

İpucu

`Ad::` içeren bir hedef adı, CMake tarafından “bu bir gerçek hedeftir” olarak katı denetlenir: yanlış yazarsanız sessizce `-lYanlis` denemek yerine hemen hata verir. Bu yüzden namespace'li adlar daha güvenlidir.

KISIM 3

Orta Seviye — Yapı, Bağımlılık ve Mantık

7. Çok Dizinli Proje Düzeni

Gerçek projeler tek bir CMakeLists.txt dosyasıyla yürütülmez. Her mantıksal bileşen kendi alt dizininde, kendi CMakeLists.txt dosyasıyla yaşar; kök dosya da bunların hepsini add_subdirectory ile bir araya getirir.

7.1 Önerilen Klasör Yapısı

```

1 myproject/
2 |-- CMakeLists.txt           # root: project-wide settings
3 |-- include/                 # public headers (optional centralized location)
4 |-- src/
5 |   '-- CMakeLists.txt       # the main application
6 |-- libs/
7 |   '-- math/
8 |       |-- CMakeLists.txt   # the math library
9 |       |-- include/math/
10 |           '-- matrix.h
11 |       '-- src/matrix.cpp
12 |-- tests/
13 |   '-- CMakeLists.txt       # tests
14 '-- build/                   # (generated; listed in .gitignore)

```

7.2 Kök CMakeLists.txt

CMakeLists.txt (kök)

```

1 cmake_minimum_required(VERSION 3.20)
2 project(MyProject VERSION 1.0 LANGUAGES CXX)
3
4 # Project-wide settings
5 set(CMAKE_CXX_STANDARD 17)
6 set(CMAKE_CXX_STANDARD_REQUIRED ON)
7
8 # Collect all executables in one place (optional, but keeps things tidy)
9 set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
10
11 # Emit compile_commands.json (very useful for clangd and IDEs)

```

```

12 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
13
14 # Add subdirectories in order (dependency order matters!)
15 add_subdirectory(libs/math)    # the library first
16 add_subdirectory(src)         # then the application that uses it
17
18 # Add the tests conditionally
19 option(BUILD_TESTS "Build the unit tests" ON)
20 if(BUILD_TESTS)
21     enable_testing()
22     add_subdirectory(tests)
23 endif()

```

7.3 Kütüphane Alt Dizini

libs/math/CMakeLists.txt

```

1 add_library(math STATIC
2     src/matrix.cpp
3     src/vector.cpp
4 )
5 add_library(MyProject::math ALIAS math)
6
7 target_include_directories(math PUBLIC
8     ${CMAKE_CURRENT_SOURCE_DIR}/include
9 )
10 target_compile_features(math PUBLIC cxx_std_17)

```

src/CMakeLists.txt

```

1 add_executable(app main.cpp)
2 target_link_libraries(app PRIVATE MyProject::math) # consume the library

```

Bilgi

add_subdirectory sırası önemlidir: bir hedefe target_link_libraries ile bağlanmadan önce o hedefin tanımlanmış olması gerekir. Kütüphaneleri her zaman kullanıcılarından önce ekleyin.

MINİ PROJE • Yukarıdaki düzeni çalışan hale getirmek

Eksik kalan tek şey kaynak dosyaları. Aşağıdaki üç dosyayı ekleyince proje uçtan uca derlenir (testleri şimdilik kapatıyoruz):

libs/math/include/math/matrix.h

```

1 #pragma once
2 namespace mp { int add(int a, int b); }

```

libs/math/src/matrix.cpp

```

1 #include "math/matrix.h"
2 namespace mp { int add(int a, int b) { return a + b; } }

```

src/main.cpp

```

1 #include <iostream>
2 #include "math/matrix.h"           // include/math yolu MyProject::math'ten geldi
3 int main() { std::cout << "2 + 3 = " << mp::add(2, 3) << "\n"; }

```

```

1 # tests/ klasorunu yazmadığımız için BUILD_TESTS=OFF veriyoruz
2 cmake -S . -B build -G Ninja -D BUILD_TESTS=OFF
3 cmake --build build
4 ./build/bin/app           # çıktı build/bin altına gider (kaktaki ayar)
5 # 2 + 3 = 5

```

8. Harici Bağımlılıklar: find_package

Çoğu proje, sistemde kurulu kütüphanelerden (Threads, OpenGL, Boost, SQLite, fmt gibi) yararlanır. find_package bu kütüphaneleri sizin için bulur ve doğrudan bağlayabileceğiniz hazır hedefler sunar.

8.1 İki Mod: Config ve Module

find_package iki şekilde arama yapar:

Mod	Açıklama
Config	Kütüphanenin kendi kurduğu XxxConfig.cmake dosyasını bulur. Modern kütüphaneler bunu sağlar (en güvenilir).
Module	CMake ile gelen veya sizin yazdığınız FindXxx.cmake bulma betiğini kullanır (eski kütüphaneler için).

8.2 Örnek: Threads (pthread)

```

1 find_package(Threads REQUIRED)           # REQUIRED: stop with an error if not found
2
3 add_executable(server server.cpp)
4 target_link_libraries(server PRIVATE Threads::Threads)

```

8.3 Örnek: fmt Kütüphanesi (Config mod)

```

1 find_package(fmt 9.0 REQUIRED)           # require version 9.0 or newer
2
3 add_executable(app main.cpp)
4 target_link_libraries(app PRIVATE fmt::fmt)

```

main.cpp

```

1 #include <fmt/core.h>
2 int main() {
3     // {} are placeholders, filled positionally by the following arguments
4     fmt::print("Hello {}! {} + {} = {}\n", "world", 2, 3, 2 + 3);
5 }

```

8.4 COMPONENTS ve Opsiyonel Kullanım

```

1 # Request only specific components of a large package
2 find_package(Boost 1.75 REQUIRED COMPONENTS filesystem system)
3 target_link_libraries(app PRIVATE Boost::filesystem Boost::system)
4
5 # Without REQUIRED: enable the feature if found, otherwise continue silently
6 find_package(OpenMP)
7 if(OpenMP_CXX_FOUND)
8     target_link_libraries(app PRIVATE OpenMP::OpenMP_CXX)
9     message(STATUS "OpenMP found; parallel build enabled")
10 else()
11     message(STATUS "OpenMP not found; building without parallelism")
12 endif()

```

Not

CMake'e kütüphanenin nerede olduğunu söylemek için `-DCMAKE_PREFIX_PATH=/opt/mylib` kullanın. `find_package` bu yolları da tarar.

8.5 Elle Kurulan Kütüphaneler: CMake Nerede Arar?

Sık karşılaşılan bir durum: bir kütüphaneyi paket yöneticisinden değil, kaynaktan derleyip `make install` (veya `cmake -install`) ile kurarsınız. Bu genellikle dosyaları `/usr/local` altına yerleştirir:

```

1 /usr/local/lib/           # .so / .a (kutuphane dosyaları)
2 /usr/local/include/       # .h / .hpp (başlıklar)
3 /usr/local/lib/cmake/Xxx/ # XxxConfig.cmake <-- find_package BUNU arar

```

`find_package(Xxx)`, kütüphane dosyasını (`.so`) değil, o kütüphanenin kurulum sırasında bıraktığı `XxxConfig.cmake` dosyasını arar. CMake bu dosyayı ararken şu *ön ek* (prefix) yollarının her birinin altına bakar:

Aranan yol (<prefix> altında)

Açıklama

<code>lib/cmake/Xxx/</code>	En yaygın konum.
<code>lib/Xxx/cmake/</code>	Bazı kütüphanelerin tercihi.
<code>share/Xxx/cmake/</code>	Mimariden bağımsız kurulumlar.
<code>lib64/cmake/Xxx/</code>	Bazı 64-bit dağıtımlar.

Buradaki `<prefix>`, CMake'in varsayılan olarak taradığı köklerdir: `CMAKE_SYSTEM_PREFIX_PATH`. Linux'ta bu liste `/usr` ve `/usr/local`'i *zaten içerir*; bu yüzden `/usr/local` altına düzgün kurulmuş çoğu kütüphane hiçbir ayar yapmadan bulunur.

Peki neden bazen bulunamıyor?

- **Farklı bir ön eke kuruldu:** Kütüphane `/usr/local` yerine `/opt/mylib` ya da `$HOME/.local` gibi standart dışı bir yere kurulmuştur (`-DCMAKE_INSTALL_PREFIX` veya `./configure -prefix`). CMake oralara varsayılan olarak bakmaz.
- **Config dosyası hiç üretilmemiş:** Kütüphane yalnızca `.so` ve başlık kurar, `XxxConfig.cmake` sağlamaz (eski veya Autotools tabanlı projeler). Bu durumda `find_package`'in Config modu için tutunacak bir şey yoktur.

- **lib / lib64 uyumsuzluğu:** Dosyalar lib64 altındayken CMake lib altına bakıyordur (dağıtıma göre değişir).

Çözüm 1 — CMAKE_PREFIX_PATH (standart dışı ön ek)

CMake'e "şu kök(ler)in altına da bak" demenin en temiz yolu budur. Verdiğiniz yolun altındaki lib/cmake/... vb. otomatik taranır:

```
1 # Kutuphane /opt/mylib altına kuruldu diyelim:
2 cmake -S . -B build -DCMAKE_PREFIX_PATH=/opt/mylib
3
4 # Birden fazla yol -- noktalı virgülle ayrılır:
5 cmake -S . -B build -DCMAKE_PREFIX_PATH="/opt/mylib;$HOME/.local"
```

Aynı şeyi kalıcı olarak ortam değişkeni ile de verebilirsiniz; CMake bunu okur:

```
1 export CMAKE_PREFIX_PATH=/opt/mylib:$HOME/.local # (: ile ayırın)
```

Çözüm 2 — Xxx_DIR (doğrudan config klasörü)

Yalnızca tek bir paket sorun çıkarıyorsa, o paketin Config.cmake dosyasının bulunduğu klasörü doğrudan gösterebilirsiniz. Değişkenin adı <PaketAdı>_DIR biçimindedir:

```
1 cmake -S . -B build \
2     -Dfmt_DIR=/opt/fmt/lib/cmake/fmt # XxxConfig.cmake'in olduğu klasör
```

Çözüm 3 — Config yoksa: pkg-config'e düşün

Kütüphane hiç XxxConfig.cmake sağlamıyorsa ama bir .pc (/usr/local/lib/pkgconfig/xxx.pc) dosyası bırakmışsa, find_package yerine pkg-config köprüsünü kullanabilirsiniz:

```
1 find_package(PkgConfig REQUIRED)
2 pkg_check_modules(XXX REQUIRED IMPORTED_TARGET xxx) # xxx = .pc dosyasının adı
3
4 add_executable(app main.cpp)
5 target_link_libraries(app PRIVATE PkgConfig::XXX) # bayraklar/yollar otomatik
```

Örnek: İki farklı yoldaki iki kütüphane

Gerçekçi bir senaryo: projeniz iki kütüphaneye bağlı. Biri kaynaktan derleyip /usr/local altına kurduğunuz kendi mylib'iniz, diğeri ise resmi kurulumla /opt/Qt/6.7.0/gcc_64 altına gelen Qt. İkisi farklı ön eklerde durduğundan, CMAKE_PREFIX_PATH'e her ikisini de vermeniz gerekir:

```
1 # Her iki paketin de config dosyaları bulunan köklerini ; ile ver:
2 cmake -S . -B build -G Ninja \
3     -DCMAKE_PREFIX_PATH="/usr/local;/opt/Qt/6.7.0/gcc_64"
4
5 # /usr/local altında -> lib/cmake/mylib/mylibConfig.cmake
6 # /opt/Qt/6.7.0/gcc_64 altında -> lib/cmake/Qt6/Qt6Config.cmake
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.20)
2 project(İkiKutuphane LANGUAGES CXX)
3 set(CMAKE_CXX_STANDARD 17)
4
5 # find_package her iki koku de CMAKE_PREFIX_PATH sayesinde tarar:
6 find_package(mylib REQUIRED) # /usr/local altından
7 find_package(Qt6 REQUIRED COMPONENTS Widgets) # /opt/Qt/... altından
```

```

8
9 add_executable(app main.cpp)
10 target_link_libraries(app PRIVATE
11     mylib::mylib           # kendi kutuphaneniz
12     Qt6::Widgets)         # Qt bileşeni

```

İpucu

Yalnızca Qt sorun çıkarıyorsa, CMAKE_PREFIX_PATH yerine tek bir paketi hedefleyen -DQt6_DIR=/opt/Qt/6.7.0/gcc_64/lib/cmake/Qt6 da işe yarar. CMAKE_PREFIX_PATH birden çok paket için, _DIR tek bir paket için pratiktir.

Dikkat

/usr/local/lib'e kurdunuz ama çalıştırırken error while loading shared libraries: libxxx.so alıyorsanız bu bir CMake sorunu değildir: dinamik yükleyici bu klasörü taramıyordu. /etc/ld.so.conf.d/ altına yolu ekleyip sudo ldconfig çalıştırın veya export LD_LIBRARY_PATH=/usr/local/lib verin.

İpucu

Paket gerçekten nerede aranıyor, göremiyor musunuz? cmake -debug-find -S . -B build komutu find_package'in denediği tüm yolları tek tek yazdırır; hangi klasöre bakıp bulamadığını anında görürsünüz.

MİNİ PROJE • find_package(Threads) ile çalışan bir çok-iş-parçacıklı program

Threads, hemen her sistemde hazır bulunur; ayrıca bir şey kurmanız gerekmez. Bu yüzden find_package'i denemenin en kolay yolu budur.

```

1 threads-demo/
2 |-- CMakeLists.txt
3 '-- main.cpp

```

main.cpp

```

1 #include <iostream>
2 #include <thread>
3 #include <vector>
4
5 int main() {
6     std::vector<std::thread> workers;
7     for (int i = 0; i < 3; ++i)
8         workers.emplace_back([i]{ std::cout << "Is parçacigi " << i << "\n"; });
9     for (auto& t : workers) t.join(); // hepsinin bitmesini bekle
10 }

```

CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.20)
2 project(ThreadsDemo LANGUAGES CXX)
3 set(CMAKE_CXX_STANDARD 17)
4
5 # Sistemdeki is parçacigi destegini bul (Linux'ta pthread'e çözülür)
6 find_package(Threads REQUIRED)

```

```

7
8 add_executable(app main.cpp)
9 target_link_libraries(app PRIVATE Threads::Threads) # -pthread'i CMake ekler

1 cmake -S . -B build -G Ninja && cmake --build build
2 ./build/app
3 # Is parcacigi 0
4 # Is parcacigi 1 # sıra karisabilir -- is parcaciklari es zamanli calisir
5 # Is parcacigi 2

```

9. Fonksiyonlar ve Makrolar

Bir CMake fonksiyonu, tıpkı herhangi bir programlama dilindeki fonksiyon gibi, tekrar eden bir işi *bir kez* tanımlayıp *defalarca* çağırmanızı sağlar. Aynı beş satırlık hedef kurulumunu on ayrı yere kopyalamak yerine, o mantığı bir fonksiyonun içine koyar ve her seferinde tek satırla çağırırsınız. Bu bölümde önce bir fonksiyonun yapısını, ardından *gerçek bir projede* onu nasıl tanımlayıp kullanacağınızı adım adım ele alacağız.

9.1 function vs macro

function	macro
Kendi değişken kapsamı vardır (yereldir).	Kapsamı yoktur; çağırıldığı yere metin gibi genişler.
PARENT_SCOPE ile dışarı yazar.	Çağıranın değişkenlerini doğrudan değiştirir.
Genelde tercih edilir (daha güvenli).	Sadece değişken sızdırmak gerektiğinde.

9.2 Bir Fonksiyonun Anatomisi: Parametreler ve ARGN

Bir fonksiyonu `function(<isim> <par1> <par2> ...)` ile tanımlarsınız. Adlandırılmış parametrelerin yanı sıra CMake, her çağırıda birkaç **otomatik değişkeni** de sizin için doldurur: ARGC (argüman sayısı), ARGV (tüm argümanlar, tek bir liste olarak), ARGN (adlandırılmış parametrelerden *sonra* gelen “fazladan” argümanlar) ve ARGVO, ARGV1, ... (indeksle erişim). Bunlardan ARGN özellikle önemlidir: değişken sayıda kaynak dosyası alan hedef kurucular tam da bunun üzerine inşa edilir.

```

1 function(demo first second)
2     # Named positional parameters:
3     message(STATUS "first = ${first}") # 1st argument
4     message(STATUS "second = ${second}") # 2nd argument
5
6     # Automatic variables available inside every function:
7     message(STATUS "ARGC = ${ARGC}") # total number of arguments
8     message(STATUS "ARGV = ${ARGV}") # ALL arguments, as a ;-list
9     message(STATUS "ARGN = ${ARGN}") # only the EXTRA args (after first, second)
10    message(STATUS "ARGVO = ${ARGVO}") # index access -> same as 'first'
11 endfunction()
12
13 demo(one two three four)
14 # -- first = one
15 # -- second = two
16 # -- ARGC = 4

```

```

17 # -- ARGV = one;two;three;four
18 # -- ARGV = three;four
19 # -- ARGV = one

```

Bilgi

Bir fonksiyon *kendi değişken kapsamına* sahiptir: içinde `set(x ...)` yaptığınızda bu `x`, fonksiyon bittiğinde yok olur ve çağırının `x`'ini kirletmez. Bu yüzden hedef kurmak gibi işler için fonksiyon, makrodan daha güvenlidir.

9.3 Neden Fonksiyon? Önce/Sonra

Diyelim ki projenizde bir `examples/` klasörü var ve içinde onlarca küçük örnek program bulunuyor. Bunların her biri *aynı* kütüphaneye bağlanmalı, *aynı* C++ standardını ve *aynı* uyarı bayraklarını kullanmalı, çıktısı da *aynı* klasöre düşmeli. Fonksiyon kullanmazsanız bu, her örnek için tekrar tekrar yazılan dört satır demektir:

`examples/CMakeLists.txt -- fonksiyonsuz (tekrar dolu)`

```

1 add_executable(gauss gauss.cpp)
2 target_link_libraries(gauss PRIVATE MyProject::math project_warnings)
3 target_compile_features(gauss PRIVATE cxx_std_17)
4 set_target_properties(gauss PROPERTIES RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/examples
5 )
6 add_executable(fourier fourier.cpp fft.cpp)
7 target_link_libraries(fourier PRIVATE MyProject::math project_warnings)
8 target_compile_features(fourier PRIVATE cxx_std_17)
9 set_target_properties(fourier PROPERTIES RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/
10 examples)
11 # ... 10 örnek daha, her biri aynı 4 satır -> bakımı imkansız, hataya açık

```

Bu mantığı tek bir `add_example` fonksiyonuna taşırsak, aynı iş her örnek için **tek satıra** iner:

`examples/CMakeLists.txt -- fonksiyonlu (temiz)`

```

1 add_example(gauss gauss.cpp) # tek kaynak
2 add_example(fourier fourier.cpp fft.cpp) # birden çok kaynak (ARGN'e gider)
3 add_example(newton newton.cpp)

```

9.4 Gerçek Proje: Fonksiyonu Ayrı Bir Modüle Koymak

Profesyonel projelerde yardımcı fonksiyonlar kök `CMakeLists.txt` içine değil, `cmake/` klasöründeki ayrı bir `.cmake` modülüne yazılır; kök dosya da bu modülü `include()` ile yükler. Böylece fonksiyon tüm projede, yani bütün alt dizinlerde kullanılabilir hale gelir. Aşağıda bunun uçtan uca tam bir örneği var.

Proje düzeni

```

1 myproject/
2 |-- CMakeLists.txt # kök: modülü yükler, alt dizinleri ekler
3 |-- cmake/
4 |   '-- AddExample.cmake # yeniden kullanılabilir yardımcı fonksiyon
5 |-- libs/math/ # MyProject::math kütüphanesi
6 '-- examples/
7   |-- CMakeLists.txt # sadece add_example(...) çağrılarını
8   |-- gauss.cpp

```

```

9      |-- fourier.cpp
10     '-- fft.cpp

```

Önce fonksiyonun *tanımı* kendi modül dosyasına yazılır:

cmake/AddExample.cmake

```

1  # Reusable helper: build ONE example program, already wired to the math
2  # library, the shared warning flags, and a common output directory.
3  include_guard(GLOBAL)    # run this file's body at most once, even if included twice
4
5  function(add_example name)
6      # name -> the target (executable) name, e.g. "gauss"
7      # ARGV -> every remaining argument = this example's source files
8      add_executable(${name} ${ARGV})
9
10     target_link_libraries(${name} PRIVATE
11         MyProject::math      # our own library (namespaced ALIAS target)
12         project_warnings     # shared warning policy (an INTERFACE target)
13     )
14     target_compile_features(${name} PRIVATE cxx_std_17)
15
16     set_target_properties(${name} PROPERTIES
17         RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/examples
18     )
19
20     # Bonus: also register the example as a smoke test -- running it must
21     # succeed (exit code 0). One function call now gives us build + test.
22     add_test(NAME example_${name} COMMAND ${name})
23 endfunction()

```

Kök dosya, CMake'e "kendi modüllerimi cmake/ klasöründe ara" dedikten sonra fonksiyonu include ile yükler. Bu include çağrısından *sonra* eklenen her alt dizin, artık bu fonksiyona erişebilir:

CMakeLists.txt (kök) -- ilgili kısım

```

1  cmake_minimum_required(VERSION 3.20)
2  project(MyProject VERSION 1.0 LANGUAGES CXX)
3
4  # 1) CMake'in modül arama yoluna kendi cmake/ klasörümüzü ekle
5  list(APPEND CMAKE_MODULE_PATH ${CMAKE_SOURCE_DIR}/cmake)
6
7  # 2) Modülü yükle -> artık add_example() her yerde tanımlı
8  include(AddExample)    # AddExample.cmake dosyasını çalıştırır
9
10 enable_testing()        # add_test() çağrılarının kaydedilmesi için
11
12 add_subdirectory(libs/math) # önce kütüphane (add_example onu kullanıyor)
13 add_subdirectory(examples)  # sonra örnekler

```

Son olarak alt dizin, tüm o karmaşayı unutup sadece *ne* istediğini yazar:

examples/CMakeLists.txt

```

1  add_example(gauss    gauss.cpp)          # -> hedef "gauss" + test "example_gauss"
2  add_example(fourier  fourier.cpp fft.cpp) # fazladan kaynak ARGV ile gelir
3  add_example(newton   newton.cpp)

```

Ne oluyor?

`add_example(fourier fourier.cpp fft.cpp)` satırı çalıştığında CMake, sanki o dört satırı `examples/CMakeLists.txt` içine elle yazmışsınız gibi davranır: `name = "fourier"`, `ARGN = "fourier.cpp;fft.cpp"` olur; `add_executable` bu iki kaynağı derler, kütüphane ve uyarılar bağlanır, çıktı `build/examples/` altına konur ve bir de `example_fourier` testi kaydedilir. Onlarca örnek için bakılacak tek yer artık `AddExample.cmake` dosyasıdır.

Bilgi

Bir fonksiyon *bir kez* (global olarak) tanımlanır ve tanımlandığı andan itibaren **tüm alt dizinlerde** görünür — değişkenlerin aksine, alt dizine “kopyalanması” gerekmez. Bu yüzden yardımcıları kökte `include` etmek, onları tek merkezden tüm projeye yaymanın yoludur.

İpucu

Yeniden kullanılabilir fonksiyonları `cmake/` klasöründe ayrı `.cmake` dosyalarında toplayın (`AddExample.cmake`, `Warnings.cmake`...). `include_guard(GLOBAL)` ile dosya iki kez `include` edilse bile fonksiyon “already defined” hatası vermez.

9.5 İsimli Argümanlar: `cmake_parse_arguments`

Yukarıdaki `add_example` basit bir örnekti; çünkü fazladan gelen tüm argümanları (`ARGN`) doğrudan kaynak dosyası sayıyordu. Ama fonksiyonunuzun *seçenekler* (bayraklar), *tek değerli* ve *çok değerli* anahtarlar alması gerekiyorsa, `ARGN`’i elle ayırtırmak epey zahmetlidir. CMake’in yerleşik `cmake_parse_arguments` komutu bu işi sizin yerinize yapar: her anahtarı `<önek>_<ANAHTAR>` biçiminde bir değişkene yerleştirir.

```

1 function(add_app name)
2     # cmake_parse_arguments(<prefix> <options> <one_value> <multi_value> ${ARGN})
3     # options    "GUI"          -> ARG_GUI          TRUE/FALSE (bayrak var mı?)
4     # one_value  "VERSION"      -> ARG_VERSION     tek bir değer
5     # multi      "SOURCES;LIBS" -> ARG_SOURCES / ARG_LIBS birer liste
6     cmake_parse_arguments(ARG "GUI" "VERSION" "SOURCES;LIBS" ${ARGN})
7
8     add_executable(${name} ${ARG_SOURCES})
9     if(ARG_LIBS)
10         target_link_libraries(${name} PRIVATE ${ARG_LIBS})
11     endif()
12     if(ARG_VERSION)
13         target_compile_definitions(${name} PRIVATE APP_VERSION="${ARG_VERSION}")
14     endif()
15     if(ARG_GUI)
16         target_compile_definitions(${name} PRIVATE USE_GUI)
17     endif()
18 endfunction()
19
20 # Çağrı: anahtarların sırası önemli değildir, okunması kolaydır
21 add_app(app
22     GUI          # bir bayrak (değer almaz)
23     VERSION 1.4  # tek değerli anahtar
24     SOURCES main.cpp ui.cpp # çok değerli anahtar
25     LIBS math Threads::Threads # çok değerli anahtar
26 )

```

Not

Böyle bir imza, çağrı yerinde kodun *niyetini* okunur kılar: `add_app(app GUI VERSION 1.4 SOURCES ... LIBS ...)` satırına bakan biri, argümanların ne anlama geldiğini sırayı ezberlemekten anlar. Büyük projelerde kendi hedef kurucularınızı hep bu kalıpla yazmak, `CMakeLists.txt` dosyalarını okunur tutar.

MINİ PROJE • Küçük ama çalışan bir `add_example` fonksiyonu

Yukarıdaki örnek büyük bir projeyi anlatıyordu. İşte fikri kanıtlayan, hemen derlenebilen en küçük hali: tek fonksiyon, iki örnek program.

```
1 funcdemo/
2 |-- CMakeLists.txt
3 |-- hello.cpp
4 '-- bye.cpp
```

hello.cpp / bye.cpp

```
1 // hello.cpp
2 #include <iostream>
3 int main() { std::cout << "Merhaba fonksiyon!\n"; }
4
5 // bye.cpp
6 #include <iostream>
7 int main() { std::cout << "Güle güle!\n"; }
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.20)
2 project(FuncsDemo LANGUAGES CXX)
3
4 # Tekrar eden hedef kurulumunu tek bir fonksiyona sariyoruz
5 function(add_example name)
6     add_executable(${name} ${ARGN})      # ARGN = fazladan gelen kaynak dosyalar
7     target_compile_features(${name} PRIVATE cxx_std_17)
8     set_target_properties(${name} PROPERTIES
9         RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/examples)
10 endfunction()
11
12 # Artık her örnek tek satır:
13 add_example(hello hello.cpp)
14 add_example(bye bye.cpp)
```

```
1 cmake -S . -B build -G Ninja && cmake --build build
2 ./build/examples/hello      # Merhaba fonksiyon!
3 ./build/examples/bye       # Güle güle!
```

10. Generator Expressions

Generator expression (kısaca “genex”), `$<...>` biçiminde yazılan ve *yapılandırma anında değil, üretim anında* değerlendirilen ifadelerdir. Derleyiciye, derleme tipine (Debug/Release) ya da hedefin özelliklerine göre farklı değerler üretmenin tek doğru yolu bunlardır.

10.1 Neden Gerekir?

Bir `if(CMAKE_BUILD_TYPE STREQUAL "Debug")` kontrolü, çok yapılandırılmalı generator'larda (Visual Studio, Xcode) işe yaramaz; çünkü oralarda derleme tipi *derleme anında* seçilir, yapılandırma anında henüz belli değildir. Genex tam da bu sorunu çözer.

10.2 Derleyiciye Göre Bayrak

```
1 add_executable(app main.cpp)
2
3 target_compile_options(app PRIVATE
4     # these flags only when the compiler is GCC or Clang
5     $<$<CXX_COMPILER_ID:GNU,Clang>:-Wall -Wextra -Wpedantic>
6     # this flag only when the compiler is MSVC
7     $<$<CXX_COMPILER_ID:MSVC>:/W4>
8 )
```

10.3 Derleme Tipine Göre Tanım

```
1 target_compile_definitions(app PRIVATE
2     # defined only in Debug builds
3     $<$<CONFIG:Debug>:DEBUG_MODE ENABLE_ASSERTS>
4     # defined only in Release builds
5     $<$<CONFIG:Release>:NDEBUG FAST_MATH>
6 )
```

10.4 Sık Kullanılan Genex Kalıpları

İfade	Anlamı
<code>\$<CONFIG:Debug></code>	Derleme tipi Debug ise 1, değilse 0.
<code>\$<CXX_COMPILER_ID:GNU></code>	Derleyici GCC ise 1.
<code>\$<PLATFORM_ID:Windows></code>	Hedef platform Windows ise 1.
<code>\$<BUILD_INTERFACE:...></code>	Sadece derleme ağacında (install değil).
<code>\$<INSTALL_INTERFACE:...></code>	Sadece kurulumdan sonra.
<code>\$<TARGET_FILE:app></code>	Hedefin üretilen dosya yolu.

10.5 BUILD_INTERFACE / INSTALL_INTERFACE

Bir kütüphane kurulduğunda include yolu değişir: kaynak ağacındaki konumdan kurulum dizinine kayar (→). Bu ikili genex, her iki durumu da doğru şekilde ele alır; kendi paketinizi yazarken (Kısım 4) bu ayrım kritiktir.

```
1 target_include_directories(math PUBLIC
2     $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include> # while building here
3     $<INSTALL_INTERFACE:include> # after installation
4 )
```


MINİ PROJE • Genex'in etkisini gözle görmek: Debug ile Release farklı davranışın

Aşağıdaki program, derleme tipine göre *farklı* bir mesaj yazar — çünkü `DEBUG_MODE` makrosunu yalnızca Debug yapıya genex ile ekliyoruz. Kaynak kod hiç değişmeden, çıktı yapıya göre değişir.

main.cpp

```
1 #include <iostream>
2 int main() {
3 #ifdef DEBUG_MODE
4     std::cout << "Debug yapisi: ayrintili gunluk acik\n";
5 #else
6     std::cout << "Release yapisi: hizli mod\n";
7 #endif
8 }
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.20)
2 project(GenexDemo LANGUAGES CXX)
3
4 add_executable(app main.cpp)
5
6 # DEBUG_MODE makrosu SADECE Debug yapida tanimlanir
7 target_compile_definitions(app PRIVATE ${CONFIG:Debug}:DEBUG_MODE)
8
9 # Uyari bayraklari SADECE GCC/Clang'te eklenir (MSVC bunlari reddederdi)
10 target_compile_options(app PRIVATE
11     ${CXX_COMPILER_ID:GNU,Clang}:-Wall;-Wextra)
```

```
1 # Ayni kaynak, iki farkli yapi -> iki farkli cikti
2 cmake -S . -B build-dbg -G Ninja -D CMAKE_BUILD_TYPE=Debug
3 cmake --build build-dbg && ./build-dbg/app
4 # Debug yapisi: ayrintili gunluk acik
5
6 cmake -S . -B build-rel -G Ninja -D CMAKE_BUILD_TYPE=Release
7 cmake --build build-rel && ./build-rel/app
8 # Release yapisi: hizli mod
```

11. Derleme Tipleri, Uyarılar ve Sanitizer'lar

11.1 CMAKE_BUILD_TYPE

Tek yapılandırılmalı generator'larda (Make, Ninja) derleme tipini `CMAKE_BUILD_TYPE` değişkeni belirler. Her tip, kendine özgü optimizasyon ve hata ayıklama bayraklarıyla gelir.

Tip	Amaç	Tipik bayraklar
Debug	Hata ayıklama; optimizasyon yok, sembol var.	-g -O0
Release	Üretim; tam optimizasyon.	-O3 -DNDEBUG
RelWithDebInfo	Optimize + sembolü (profil/crash için).	-O2 -g -DNDEBUG
MinSizeRel	En küçük ikili.	-Os -DNDEBUG

```
1 cmake -S . -B build -G Ninja -D CMAKE_BUILD_TYPE=Release
2 cmake --build build
```

İpucu

Kullanıcı bir tip belirtmezse boş kalır (optimizasyonsuz). Kök dosyanıza makul bir varsayılan koymak iyidir:

```
1 if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
2     set(CMAKE_BUILD_TYPE "RelWithDebInfo" CACHE STRING "" FORCE)
3 endif()
```

11.2 Uyarıları Merkezi Açmak

Uyarıları tüm projeye dağıtmak yerine, bir INTERFACE hedefinde toplayıp yalnızca istediğiniz hedeflere bağlamak modern bir yaklaşımdır.

```
1 # A virtual (INTERFACE) target that carries the "warning policy"
2 add_library(project_warnings INTERFACE)
3 target_compile_options(project_warnings INTERFACE
4     $<$<CXX_COMPILER_ID:GNU,Clang>:-Wall;-Wextra;-Wpedantic;-Wshadow>
5     $<$<CXX_COMPILER_ID:MSVC>:/W4>
6 )
7
8 # Apply it to any target you like:
9 target_link_libraries(app PRIVATE project_warnings)
```

11.3 Sanitizer'lar (ASan, UBSan)

Bellek hatalarını ve tanımsız davranışları yakalamakta sanitizer'ların değeri ölçülemez. Bunları bir seçeneğin arkasına alalım.

```
1 option(ENABLE_ASAN "Address + UB sanitizer" OFF)
2
3 if(ENABLE_ASAN)
4     add_compile_options(-fsanitize=address,undefined -fno-omit-frame-pointer)
5     add_link_options(-fsanitize=address,undefined)
6 endif()
```

```
1 cmake -S . -B build -G Ninja -D ENABLE_ASAN=ON -D CMAKE_BUILD_TYPE=Debug
2 cmake --build build
3 ./build/app # prints a detailed report if a memory error occurs
```

MİNİ PROJE • Sanitizer bir hatayı yakalarken görmek

Kasten hatalı bir program yazalım: dizinin sınırının bir dışına yazıyoruz. Normalde bu hata sessizce geçebilir; ASan ise programı durdurup tam yerini gösterir.

main.cpp

```
1 int main() {
2     int a[3] = {1, 2, 3};
3     a[3] = 42;          // HATA: geçerli indeksler 0..2, a[3] sınırı asıyor
4     return a[0];
5 }
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.20)
2 project(AsanDemo LANGUAGES CXX)
3
4 option(ENABLE_ASAN "Address + UB sanitizer" OFF)
5
6 add_executable(app main.cpp)
7 if(ENABLE_ASAN)
8     target_compile_options(app PRIVATE -fsanitize=address,undefined -g)
9     target_link_options(app PRIVATE -fsanitize=address,undefined)
10 endif()
```

```
1 cmake -S . -B build -G Ninja -D ENABLE_ASAN=ON -D CMAKE_BUILD_TYPE=Debug
2 cmake --build build
3 ./build/app
4 # main.cpp:3:10: runtime error: index 3 out of bounds for type 'int [3]'
5 # ==12345==ERROR: AddressSanitizer: stack-buffer-overflow ...
6 # WRITE of size 4 ... #0 in main main.cpp:3 <- hatanın tam satiri
```

İpucu

Sanitizer'lar programı biraz yavaşlatır, o yüzden yalnızca geliştirme ve test yaparken (-D ENABLE_ASAN=ON) açın; yayınladığınız Release yapıda kapalı kalsın.

KISIM 4

İleri Seviye — Bağımlılık Çekme, Test, Paketleme

12. FetchContent: Bağımlılığı Projeye Çekmek

`find_package`, sistemde zaten kurulu olan kütüphaneleri bulur. Peki ya kullanıcının kütüphaneyi önceden kurmuş olmasını istemiyorsanız? `FetchContent`, bir bağımlılığı derleme sırasında indirir ve doğrudan projenize katar; kullanıcı yalnızca `cmake` çalıştırır, gerisini CMake halleder.

```

1 include(FetchContent)
2
3 # Declare the dependency (git repository + a fixed tag)
4 FetchContent_Declare(
5     fmt
6     GIT_REPOSITORY https://github.com/fmtlib/fmt.git
7     GIT_TAG         10.2.1          # always pin a fixed version/tag!
8 )
9
10 # Download, configure and make the dependency's targets available
11 FetchContent_MakeAvailable(fmt)
12
13 add_executable(app main.cpp)
14 target_link_libraries(app PRIVATE fmt::fmt) # just like a find_package target

```

Dikkat

`GIT_TAG` olarak `main/master` gibi hareketli dallar *kullanmayın*. Her zaman sabit bir sürüm etiketi (10.2.1) veya commit hash'i verin — aksi halde derleme tekrarlanabilir olmaz ve bir gün beklenmedik şekilde kırılır.

12.1 FetchContent + find_package Hibrit (3.24+)

CMake 3.24 ile birlikte bir bağımlılık önce sistemde `find_package` ile aranabilir, bulunamazsa indirilebilir. Böylece iki yaklaşımın da en iyi yanını bir arada kullanmış olursunuz.

```

1 FetchContent_Declare(
2     fmt
3     GIT_REPOSITORY https://github.com/fmtlib/fmt.git
4     GIT_TAG         10.2.1
5     FIND_PACKAGE_ARGS 10 # try find_package(fmt 10) first; download if missing
6 )

```

```
7 FetchContent_MakeAvailable(fmt)
```

Not

FetchContent bağımlılığı sizinle birlikte *kaynak koddan* derler — ilk derleme yavaş olabilir ama tam kontrol ve sürüm garantisi verir. Büyük bağımlılıklar (ör. Qt) için sistem paketi + find_package genelde daha pratiktir.

MINİ PROJE • fmt kütüphanesini indirip kullanan tam proje

Bu projeyi kurmak için sistemde fmt kurulu olması gerekmez; CMake ilk derlemede onu GitHub'dan çeker (bu adım için internet gerekir).

```
1 fetch-demo/
2 |-- CMakeLists.txt
3 '--- main.cpp
```

main.cpp

```
1 #include <fmt/core.h>
2 int main() {
3     // {} yer tutucuları, sonraki argumanlarla sırayla doldurulur
4     fmt::print("Merhaba {}! {} + {} = {}\n", "fmt", 2, 3, 2 + 3);
5 }
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.20)
2 project(FetchDemo LANGUAGES CXX)
3 set(CMAKE_CXX_STANDARD 17)
4
5 include(FetchContent)
6 FetchContent_Declare(fmt
7     GIT_REPOSITORY https://github.com/fmtlib/fmt.git
8     GIT_TAG         10.2.1)      # her zaman sabit sürüm etiketi ver
9 FetchContent_MakeAvailable(fmt)  # indir, yapilandir, hedeflerini hazir et
10
11 add_executable(app main.cpp)
12 target_link_libraries(app PRIVATE fmt::fmt)
```

```
1 cmake -S . -B build -G Ninja    # fmt bu asamada indirilir (ilk sefer yavas)
2 cmake --build build
3 ./build/app
4 #   Merhaba fmt! 2 + 3 = 5
```

13. Test: CTest ile

CMake, kendi test koçucusu ctest ile birlikte gelir. Testlerinizi kaydeder, çalıştırır ve sonuçlarını raporlar.

13.1 Temel Test Kaydı

kök CMakeLists.txt

```

1 enable_testing()           # enable testing (call once, in the root list file)
2 add_subdirectory(tests)

```

tests/CMakeLists.txt

```

1 add_executable(math_test math_test.cpp)
2 target_link_libraries(math_test PRIVATE MyProject::math)
3
4 # Register a test: a name + the command to run
5 add_test(NAME MatrixMultiply COMMAND math_test)
6
7 # Decide pass/fail from the program's output instead of its exit code
8 add_test(NAME VersionCheck COMMAND app --version)
9 set_tests_properties(VersionCheck PROPERTIES
10     PASS_REGULAR_EXPRESSION "1\\.0\\.0")

```

math_test.cpp (basit iddia)

```

1 #include <cassert>
2 #include "math/matrix.h"
3
4 int main() {
5     Matrix a = Matrix::identity(3);
6     assert(a(0, 0) == 1.0 && a(0, 1) == 0.0);
7     // If an assert fails it aborts the program with a non-zero exit code,
8     // and CTest reports the test as "failed".
9     return 0;
10 }

```

```

1 cmake --build build
2 cd build && ctest --output-on-failure # run all tests, show output on failure
3 ctest -R Matrix                      # only tests whose name matches "Matrix"
4 ctest -j 8                           # run 8 tests in parallel
5 ctest --rerun-failed --output-on-failure # re-run only the tests that failed

```

13.2 GoogleTest ile Entegrasyon

```

1 include(FetchContent)
2 FetchContent_Declare(googletest
3     GIT_REPOSITORY https://github.com/google/googletest.git
4     GIT_TAG         v1.14.0)
5 FetchContent_MakeAvailable(googletest)
6
7 enable_testing()
8 add_executable(all_tests test_matrix.cpp test_vector.cpp)
9 target_link_libraries(all_tests PRIVATE MyProject::math GTest::gtest_main)
10
11 # Discover each GoogleTest TEST() case as a separate CTest test
12 include(GoogleTest)
13 gtest_discover_tests(all_tests)

```

MİNİ PROJE • Kendi kendine yeten, çalışan bir test projesi

Harici bir kütüphane olmadan, sadece assert kullanan küçük bir test. Test başarısız olursa

program sıfırdan farklı bir kodla çıkar ve CTest onu “failed” olarak işaretler.

```
1 ctest-demo/
2 |-- CMakeLists.txt
3 '-- test_math.cpp
```

test_math.cpp

```
1 #include <cassert>
2 static int add(int a, int b) { return a + b; }
3
4 int main() {
5     assert(add(2, 3) == 5);
6     assert(add(-1, 1) == 0);
7     return 0;    // tum assert'ler gecerse 0 = basari
8 }
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.20)
2 project(CTestDemo LANGUAGES CXX)
3
4 enable_testing()                # testleri etkinlestir (kokte bir kez)
5 add_executable(math_test test_math.cpp)
6 add_test(NAME AddWorks COMMAND math_test)    # bir test kaydet: ad + calistirilacak komut
```

```
1 cmake -S . -B build -G Ninja && cmake --build build
2 ctest --test-dir build --output-on-failure
3 #   Test project .../build
4 #       Start 1: AddWorks
5 #   1/1 Test #1: AddWorks ..... Passed    0.00 sec
6 #   100% tests passed, 0 tests failed out of 1
```

14. Kurulum (install) ve Paketleme (CPack)

14.1 install Komutu

install(), cmake --install çalıştırıldığında hangi dosyaların nereye kopyalanacağını belirler.

```
1 # Install the executables and libraries
2 install(TARGETS app math
3     RUNTIME DESTINATION bin          # executables -> <prefix>/bin
4     LIBRARY DESTINATION lib          # shared libs (.so/.dll) -> <prefix>/lib
5     ARCHIVE DESTINATION lib          # static libs (.a/.lib) -> <prefix>/lib
6 )
7
8 # Install the public headers
9 install(DIRECTORY libs/math/include/
10     DESTINATION include              # -> <prefix>/include
11     FILES_MATCHING PATTERN "*.h"    # copy only header files, keep the tree
12 )
```

```
1 cmake -S . -B build -D CMAKE_INSTALL_PREFIX=/opt/myproject
2 cmake --build build
3 cmake --install build                # copies the files into /opt/myproject
```

14.2 CPack ile Dağıtım Paketi

CPack, projenizi .tar.gz, .deb, .rpm, Windows kurulumu gibi paketlere dönüştürür.

```
1 # Place this after your install() rules:
2 set(CPACK_PACKAGE_NAME "MyProject")
3 set(CPACK_PACKAGE_VERSION "1.0.0")
4 set(CPACK_PACKAGE_CONTACT "tolga@example.com")
5 set(CPACK_GENERATOR "TGZ;DEB")      # which package formats to produce
6 include(CPack)                     # must be included last
```

```
1 cmake --build build
2 cd build && cpack                # produces MyProject-1.0.0-Linux.tar.gz, .deb, ...
```

MİNİ PROJE • Küçük bir programı gerçekten kurmak

Bir programı bir hedef dizine kurup, kurulan dosya ağacının nasıl görüldüğünü görelim. (Sistem dizinlerini kirletmemek için hedef olarak proje içinde bir stage/ klasörü seçiyoruz.)

main.cpp

```
1 #include <iostream>
2 int main() { std::cout << "kuruldum!\n"; }
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.20)
2 project(InstallDemo LANGUAGES CXX)
3
4 add_executable(hello main.cpp)
5
6 # hello ikilisi <prefix>/bin altına kurulsun
7 install(TARGETS hello RUNTIME DESTINATION bin)
```

```
1 cmake -S . -B build -G Ninja
2 cmake --build build
3 cmake --install build --prefix ./stage    # dosyalari ./stage altına kopyalar
4
5 ./stage/bin/hello      # kuruldum!
6 find stage             # stage
7                        # stage/bin
8                        # stage/bin/hello
```

15. Kendi Config Paketini Yazmak

Başkalarının kütüphanenizi find_package(MyProject) ile kullanabilmesi için bir “config paketi” hazırlayıp kurmanız gerekir. Bu adım, profesyonel kütüphane dağıtımının anahtarıdır.

```
1 include(GNUInstallDirs)
2 include(CMakePackageConfigHelpers)
3
4 # 1) Install the target and record it in an export set (EXPORT)
5 install(TARGETS math
6         EXPORT MyProjectTargets
```



```

7   ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
8   LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
9   INCLUDES DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
10  )
11  install(DIRECTORY libs/math/include/ DESTINATION ${CMAKE_INSTALL_INCLUDEDIR})
12
13  # 2) Install the export set as a .cmake file (with a namespace prefix)
14  install(EXPORT MyProjectTargets
15    FILE MyProjectTargets.cmake
16    NAMESPACE MyProject::
17    DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject
18  )
19
20  # 3) Generate the version file (used for version checks in find_package)
21  write_basic_package_version_file(
22    "${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfigVersion.cmake"
23    VERSION ${PROJECT_VERSION}
24    COMPATIBILITY SameMajorVersion
25  )
26
27  # 4) Generate the config file from a template and install it
28  configure_package_config_file(
29    "${CMAKE_CURRENT_SOURCE_DIR}/cmake/MyProjectConfig.cmake.in"
30    "${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfig.cmake"
31    INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject
32  )
33  install(FILES
34    "${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfig.cmake"
35    "${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfigVersion.cmake"
36    DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject
37  )

```

cmake/MyProjectConfig.cmake.in

```

1  @PACKAGE_INIT@
2  include("${CMAKE_CURRENT_LIST_DIR}/MyProjectTargets.cmake")

```

Artık başka bir proje şöyle kullanabilir:

```

1  find_package(MyProject 1.0 REQUIRED)
2  target_link_libraries(other_app PRIVATE MyProject::math)

```

16. CMakePresets.json — Yapılandırmayı Sabitlemek

Uzun `-D...` komutlarını her seferinde yeniden yazmak yerine, `CMakePresets.json` dosyasında hazır yapılandırmalar tanımlarsınız. IDE'ler de bu dosyayı okuyup aynı ayarları kullanır.

CMakePresets.json

```

1  {
2    "version": 3,
3    "configurePresets": [
4      {
5        "name": "debug",
6        "displayName": "Debug (Ninja)",
7        "generator": "Ninja",

```

```

8     "binaryDir": "${sourceDir}/build/debug",
9     "cacheVariables": {
10         "CMAKE_BUILD_TYPE": "Debug",
11         "ENABLE_ASAN": "ON"
12     }
13 },
14 {
15     "name": "release",
16     "generator": "Ninja",
17     "binaryDir": "${sourceDir}/build/release",
18     "cacheVariables": { "CMAKE_BUILD_TYPE": "Release" }
19 }
20 ],
21 "buildPresets": [
22     { "name": "debug", "configurePreset": "debug" },
23     { "name": "release", "configurePreset": "release" }
24 ]
25 }

```

```

1 cmake --preset debug           # configure with all the preset's settings
2 cmake --build --preset debug
3 cmake --list-presets          # list the available presets

```

İpucu

CMakePresets.json'u git'e ekleyin: ekipteki herkes aynı yapılandırmayı tek komutla kullanır. Kişisel ayarlar için CMakeUserPresets.json (git'e eklenmez) kullanılır.

MINİ PROJE • Tek komutla yapılandırıp derlenen bir preset projesi

```

1 preset-demo/
2 |-- CMakeLists.txt
3 |-- CMakePresets.json
4 '-- main.cpp

```

main.cpp

```

1 #include <iostream>
2 int main() { std::cout << "merhaba preset\n"; }

```

CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.20)
2 project(PresetDemo LANGUAGES CXX)
3 add_executable(app main.cpp)

```

CMakePresets.json

```

1 {
2     "version": 3,
3     "configurePresets": [
4         { "name": "debug", "generator": "Ninja",
5           "binaryDir": "${sourceDir}/build/debug",
6           "cacheVariables": { "CMAKE_BUILD_TYPE": "Debug" } }

```

```

7 ],
8 "buildPresets": [ { "name": "debug", "configurePreset": "debug" } ]
9 }

```

```

1 cmake --preset debug          # uzun -D... satiri yok; hepsi preset'ten gelir
2 cmake --build --preset debug
3 ./build/debug/app
4 #   merhaba preset

```

17. Toolchain Dosyaları ve Çapraz Derleme

Başka bir mimari için derleme — örneğin Linux'ta çalışırken ARM için çıktı üretme — bir “toolchain dosyası” aracılığıyla yapılır. Bu dosya, hedef sistemin derleyicisini ve gerekli ayarlarını tanımlar.

arm-toolchain.cmake

```

1  # Describe the target system (this also enables cross-compilation mode)
2  set(CMAKE_SYSTEM_NAME Linux)
3  set(CMAKE_SYSTEM_PROCESSOR arm)
4
5  # The cross compilers
6  set(CMAKE_C_COMPILER   arm-linux-gnueabi-gcc)
7  set(CMAKE_CXX_COMPILER arm-linux-gnueabi-g++)
8
9  # The target root directory (sysroot)
10 set(CMAKE_FIND_ROOT_PATH /usr/arm-linux-gnueabi)
11
12 # Search programs on the host, but libraries/headers only in the target sysroot
13 set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
14 set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
15 set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)

```

```

1 cmake -S . -B build-arm -G Ninja \
2     -D CMAKE_TOOLCHAIN_FILE=arm-toolchain.cmake
3 cmake --build build-arm

```

KISIM 5

Pratik — Tam Projeler ve Başvuru

18. Tam Örnek: Çok Modüllü C++ Projesi

Şimdiye kadar öğrendiğimiz her şeyi çalışan tek bir projede bir araya getirelim: bir math kütüphanesi, onu kullanan bir app ve testler.

18.1 Dosya Yapısı

```
1 calc/  
2 |-- CMakeLists.txt  
3 |-- CMakePresets.json  
4 |-- libs/math/  
5 |   |-- CMakeLists.txt  
6 |   |-- include/math/calculator.h  
7 |   '-- src/calculator.cpp  
8 |-- app/  
9 |   |-- CMakeLists.txt  
10 |   '-- main.cpp  
11 '-- tests/  
12 |   |-- CMakeLists.txt  
13 |   '-- calc_test.cpp
```

18.2 Kütüphane Kaynağı

libs/math/include/math/calculator.h

```
1 #pragma once  
2 namespace calc {  
3     double add(double a, double b);  
4     double multiply(double a, double b);  
5     long factorial(int n);  
6 }
```

libs/math/src/calculator.cpp

```
1 #include "math/calculator.h"  
2  
3 namespace calc {  
4     double add(double a, double b) { return a + b; }  
5     double multiply(double a, double b) { return a * b; }  
6     long factorial(int n) {
```

```

7     long r = 1;
8     for (int i = 2; i <= n; ++i) r *= i;
9     return r;
10 }
11 }

```

18.3 Tüm CMakeLists.txt Dosyaları

CMakeLists.txt (kök)

```

1 cmake_minimum_required(VERSION 3.21)
2 project(Calc VERSION 1.0 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
7 set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
8
9 if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
10     set(CMAKE_BUILD_TYPE "RelWithDebInfo" CACHE STRING "" FORCE)
11 endif()
12
13 add_subdirectory(libs/math)
14 add_subdirectory(app)
15
16 option(BUILD_TESTS "Build the unit tests" ON)
17 if(BUILD_TESTS)
18     enable_testing()
19     add_subdirectory(tests)
20 endif()

```

libs/math/CMakeLists.txt

```

1 add_library(math STATIC src/calculator.cpp)
2 add_library(Calc::math ALIAS math)
3
4 target_include_directories(math PUBLIC
5     $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
6     $<INSTALL_INTERFACE:include>
7 )
8 target_compile_features(math PUBLIC cxx_std_17)

```

app/CMakeLists.txt

```

1 add_executable(calc_app main.cpp)
2 target_link_libraries(calc_app PRIVATE Calc::math)

```

app/main.cpp

```

1 #include <iostream>
2 #include "math/calculator.h"
3
4 int main() {
5     std::cout << "2 + 3    = " << calc::add(2, 3) << "\n";
6     std::cout << "4 * 5    = " << calc::multiply(4, 5) << "\n";
7     std::cout << "5!      = " << calc::factorial(5) << "\n";
8     return 0;
9 }

```

tests/CMakeLists.txt

```

1 add_executable(calc_test calc_test.cpp)
2 target_link_libraries(calc_test PRIVATE Calc::math)
3 add_test(NAME CalcBasic COMMAND calc_test)

```

tests/calc_test.cpp

```

1 #include <cassert>
2 #include "math/calculator.h"
3
4 int main() {
5     assert(calc::add(2, 3) == 5.0);
6     assert(calc::multiply(4, 5) == 20.0);
7     assert(calc::factorial(5) == 120);
8     return 0;    // if every assert passes, 0 = success
9 }

```

18.4 Derleme ve Çalıştırma

```

1 cmake -S . -B build -G Ninja
2 cmake --build build
3 ./build/bin/calc_app
4 #  2 + 3  = 5
5 #  4 * 5  = 20
6 #  5!     = 120
7 cd build && ctest --output-on-failure
8 #  100% tests passed, 0 tests failed out of 1

```

19. Tam Örnek: C Projesi (Kütüphane + Uygulama)

Aynı yapıyı bu kez saf C ile kuralım: bir dinamik dizi (dynamic array) kütüphanesi ve onu kullanan bir uygulama.

19.1 Dosya Yapısı

```

1 darray/
2 |-- CMakeLists.txt
3 |-- lib/
4 |   |-- CMakeLists.txt
5 |   |-- include/darray.h
6 |   '-- src/darray.c
7 '-- app/
8     |-- CMakeLists.txt
9     '-- main.c

```

19.2 Kaynaklar

lib/include/darray.h

```

1 #ifndef DARRAY_H
2 #define DARRAY_H
3 #include <stddef.h>
4

```

```

5  typedef struct {
6      int    *data;
7      size_t size;
8      size_t capacity;
9  } DArray;
10
11 void darray_init(DArray *a);
12 void darray_push(DArray *a, int value);
13 void darray_free(DArray *a);
14
15 #endif /* DARRAY_H */

```

lib/src/darray.c

```

1  #include "darray.h"
2  #include <stdlib.h>
3
4  void darray_init(DArray *a) {
5      a->data = NULL; a->size = 0; a->capacity = 0;
6  }
7
8  void darray_push(DArray *a, int value) {
9      if (a->size == a->capacity) {
10         a->capacity = a->capacity ? a->capacity * 2 : 4;
11         a->data = realloc(a->data, a->capacity * sizeof(int));
12     }
13     a->data[a->size++] = value;
14 }
15
16 void darray_free(DArray *a) {
17     free(a->data);
18     a->data = NULL; a->size = a->capacity = 0;
19 }

```

app/main.c

```

1  #include <stdio.h>
2  #include "darray.h"
3
4  int main(void) {
5      DArray a;
6      darray_init(&a);
7      for (int i = 1; i <= 5; ++i) darray_push(&a, i * i);
8
9      printf("Size: %zu\n", a.size);
10     for (size_t i = 0; i < a.size; ++i) printf("%d ", a.data[i]);
11     printf("\n");
12
13     darray_free(&a);
14     return 0;
15 }

```

19.3 CMakeLists.txt Dosyaları

CMakeLists.txt (kök)

```

1  cmake_minimum_required(VERSION 3.20)
2  project(DArray VERSION 1.0 LANGUAGES C)

```

```

3
4 set(CMAKE_C_STANDARD 11)
5 set(CMAKE_C_STANDARD_REQUIRED ON)
6 set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
7
8 add_subdirectory(lib)
9 add_subdirectory(app)

```

lib/CMakeLists.txt

```

1 add_library(darray STATIC src/darray.c)
2 add_library(DArray::darray ALIAS darray)
3
4 target_include_directories(darray PUBLIC
5     ${CMAKE_CURRENT_SOURCE_DIR}/include
6 )
7 target_compile_features(darray PUBLIC c_std_11)
8
9 # Strict warnings only on GCC/Clang (MSVC would reject these flags)
10 target_compile_options(darray PRIVATE
11     $<$<C_COMPILER_ID:GNU,Clang>:-Wall;-Wextra>
12 )

```

app/CMakeLists.txt

```

1 add_executable(darray_app main.c)
2 target_link_libraries(darray_app PRIVATE DArray::darray)

```

```

1 cmake -S . -B build -G Ninja
2 cmake --build build
3 ./build/bin/darray_app
4 #   Size: 5
5 #   1 4 9 16 25

```

20. En İyi Uygulamalar

Yap

Neden

- ✓ target_* komutlarını kullan; global include_directories/add_definitions'dan kaçın.
- ✓ PUBLIC/PRIVATE/INTERFACE'i bilinçli seç — kullanım gereksinimleri kendiliğinden yayılsın.
- ✓ Out-of-source build yap (-B build); build/ klasörünü git'e ekleme.
- ✓ Kütüphanelere Ad::Alt ALIAS ver; find_package hedefleriyle tutarlı olsun.
- ✓ FetchContent'te daima sabit GIT_TAG (sürüm/commit) kullan.
- ✓ cmake_minimum_required sürümünü gerçekçi ve güncel tut (3.20+).
- ✓ CMAKE_EXPORT_COMPILE_COMMANDS ON ile IDE/clangd desteğini aç.
- ✓ Derleyiciye bağlı bayrakları genex (\$<...>) ile koşulla.

Yapma**Neden**

- ✗ `file(GLOB ...)` ile kaynak toplama — yeni dosya eklenince CMake yeniden çalışmaz, derleme kırılır.
- ✗ Kaynak dizinine derleme yapma (in-source build) — kaynağı kirletir.
- ✗ `link_libraries/include_directories`'i global kullanma.
- ✗ Sabit yollar (`/home/tolga/...`) yazma — taşınabilir değildir.
- ✗ `CMAKE_CXX_FLAGS`'i elle üzerine yazma — mevcut bayrakları ezersin.

Dikkat

`file(GLOB SOURCES "src/*.cpp")` cazip görünür ama tehlikelidir: CMake dosya listesini yapılandırma anında dondurur; yeni bir `.cpp` eklediğinizde CMake bunu fark etmez ve dosya derlemeye girmez. Kaynakları her zaman `add_executable/add_library` içinde *elle* listeleyin. (3.12+ ile `CONFIGURE_DEPENDS` kısmen yardımcı olsa da yine de önerilmez.)

21. Sık Karşılaşılan Hatalar ve Çözümleri

Hata / Belirti	Neden & Çözüm
undefined reference to 'foo'	Bağlayıcı sembolü bulamıyor. Fonksiyonu tanımlayan kütüphaneyi <code>target_link_libraries</code> ile ekleyin.
fatal error: xyz.h: No such file	Include yolu eksik. <code>target_include_directories</code> ile başlık dizinini ekleyin (doğru PUBLIC/PRIVATE ile).
Could not find package XYZ	<code>find_package</code> kütüphaneyi bulamadı. <code>-DCMAKE_PREFIX_PATH=...</code> verin veya kütüphaneyi kurun.
CMake Error: ... does not appear to contain CMakeLists.txt	<code>-S</code> yanlış dizini gösteriyor. Kök <code>CMakeLists.txt</code> 'nin bulunduğu dizini verin.
Değişiklik derlemeye yansımıyor	Eski cache. <code>build/</code> klasörünü silin ya da <code>cmake --fresh</code> ile yeniden yapılandırın.
<code>target_link_libraries ...</code> is not a target	Hedef henüz tanımlanmamış. <code>add_subdirectory</code> sırasını düzeltin (kütüphane önce).
Release'te optimizasyon yok	<code>CMAKE_BUILD_TYPE</code> boş. <code>-DCMAKE_BUILD_TYPE=Release</code> verin.

22. Hızlı Başvuru (Cheat Sheet)

22.1 En Sık Komutlar

Komut	Ne yapar
<code>cmake_minimum_required(VERSION x)</code>	Gereken en düşük CMake sürümü (ilk satır).
<code>project(Ad VERSION x LANGUAGES CXX C)</code>	Projeyi ve dilleri tanımlar.
<code>add_executable(ad kaynaklar)</code>	Çalıştırılabilir hedef.
<code>add_library(ad STATIC/SHARED src)</code>	Kütüphane hedefi.
<code>target_link_libraries(t PUBLIC x)</code>	Hedefe kütüphane bağlar (özellikler yayılır).
<code>target_include_directories(t PUBLIC d)</code>	Başlık arama yolu.
<code>target_compile_features(t PUBLIC cxx_std_17)</code>	Dil standardı gereksinimi.
<code>target_compile_definitions(t PRIVATE M)</code>	Önişlemci makrosu.
<code>find_package(X REQUIRED)</code>	Harici kütüphane bulur.
<code>add_subdirectory(dir)</code>	Alt dizindeki CMakeLists'i dahil eder.
<code>install(TARGETS ...)</code>	Kurulum kuralları.
<code>enable_testing() / add_test(...)</code>	CTest testleri.

22.2 En Sık Terminal Komutları

```

1  # Configure + generate (Ninja, Release)
2  cmake -S . -B build -G Ninja -D CMAKE_BUILD_TYPE=Release
3
4  # Build (with 8 parallel jobs)
5  cmake --build build -j 8
6
7  # Build a single target only
8  cmake --build build --target calc_app
9
10 # Run the tests
11 ctest --test-dir build --output-on-failure
12
13 # Install
14 cmake --install build --prefix /opt/calc
15
16 # Reconfigure from scratch (wipe the cache)
17 cmake --fresh -S . -B build
18 # or:  rm -rf build
19
20 # Using a preset
21 cmake --preset debug && cmake --build --preset debug

```

22.3 Önemli Yerleşik Değişkenler

Değişken	Anlamı
CMAKE_SOURCE_DIR	En üst kaynak dizini.
CMAKE_BINARY_DIR	En üst derleme dizini.
CMAKE_CURRENT_SOURCE_DIR	İçinde bulunulan CMakeLists'in dizini.
PROJECT_NAME / PROJECT_VERSION	En yakın project() değerleri.
CMAKE_BUILD_TYPE	Debug/Release/... (tek-config generator).
CMAKE_INSTALL_PREFIX	Kurulum kök dizini.
BUILD_SHARED_LIBS	Tip belirtilmeyen kütüphaneler SHARED mı?
CMAKE_CXX_STANDARD	Varsayılan C++ standardı.

İpucu

Bu rehber seni derleme sistemlerinin *neden* var olduğundan başlayıp, CMake'in hedef-tabanlı modern stiliyle çok modüllü, testli, paketlenabilir projeler kurmaya taşıdı: derleyici → make → CMake → hedefler → bağımlılıklar → test → paketleme. Buradan sonrası pratik: küçük bir kütüphane yaz, onu find_package'le tüketir, CI'a bağla. Her CMakeLists.txt satırını "bu ayar hangi hedefe ait?" sorusuyla oku — modern CMake'in tamamı bu sorunun cevabıdır. İyi derlemeler!